Interactive Query over Multiple Datasets

Qihang Liu[1], Weicheng Wang[1], Min Xie[2], Yu-Xuan Qiu[1], Rui Mao[3]
[1]Beijing Institute of Technology, Zhuhai [2]Shenzhen Institute of Computing Sciences [3]Shenzhen University
{3220252983, wangweic, qiu}@bit.edu.cn, xiemin@sics.ac.cn, mao@szu.edu.cn

***Abstract*—The interactive query is a representative operator for multi-criteria decision-making. It interacts with users to learn their preferences and returns their favorite tuples based on the learned preferences. However, existing interactive algorithms are restricted to single-dataset scenarios, where all tuples are placed in a single dataset and the goal is to identify the user's global favorite tuples from this dataset. In many practical applications, tuples can be grouped into multiple independent datasets, e.g., people can be divided into demographic groups and products can be divided into different brands. Users may want to find their local favorite tuples from each group. Motivated by this, we formalize a novel problem: Interactive Query over Multiple Datasets (IMD).**

To solve the problem, we exploit the cross-dataset preference consistency and develop two lines of algorithms with different levels of information sharing. First, we propose a *shared utility range* (SUR) mechanism, which propagates learned preference across datasets. We extend existing algorithms to their multi-dataset variants by equipping them with this mechanism. Second, we introduce a *shared-question* mechanism that selects questions informative for multiple datasets simultaneously. To further address the scalability bottleneck, we propose a localized strategy that identifies promising tuples in each dataset and formulates shared questions over them. Combining this strategy with the shared-question mechanism yields an effective algorithm. Extensive experiments show that our algorithms reduce the number of questions by 50% to 90% while maintaining efficiency.

***Index Terms*—Interactive Query, Multiple Datasets, Preference Elicitation, Multi-Criteria Decision Making**

I. Introduction

Multi-criteria decision-making [1], [2], [3], [4], [5] aims to help users identify desired tuples from large-scale databases. It has been widely used in applications, e.g., apartment rentals, job seeking, and car purchases. One representative operator for multi-criteria decision-making is the interactive query [6], [7], [8], [9], [10], which interacts with users to learn their preferences and returns tuples based on the learned preferences.

Intuitively, the interactive query models the user's preference by a *linear utility function* $f_{\boldsymbol{u}}(\boldsymbol{p}) = \boldsymbol{u} \cdot \boldsymbol{p}$, where $\boldsymbol{p}$ denotes a multi-attribute tuple and $\boldsymbol{u}$ is a multi-dimensional vector, called *utility vector* [11], [12], [13], [14]. Each dimension of $\boldsymbol{u}$ captures the importance of an attribute of $\boldsymbol{p}$ in the user's preference, and a larger function score indicates that $\boldsymbol{p}$ is more preferred. To learn the user's preference, i.e., the specific $\boldsymbol{u}$, the interactive query asks the user to make a sequence of pairwise comparisons [15], [16]: *Do you prefer tuple $\boldsymbol{p}$ or tuple $\boldsymbol{q}$?* Each feedback induces a constraint on $\boldsymbol{u}$, e.g., if the user prefers $\boldsymbol{p}$ to $\boldsymbol{q}$, then $f_{\boldsymbol{u}}(\boldsymbol{p}) > f_{\boldsymbol{u}}(\boldsymbol{q})$, i.e., $\boldsymbol{u} \cdot (\boldsymbol{p} - \boldsymbol{q}) > 0$. By accumulating such constraints, the feasible range of $\boldsymbol{u}$ is gradually narrowed. Once sufficiently narrow, the user's preference can be inferred, and the user's favorite tuples can be returned accordingly.

Table I: Car Dataset

Car	Horsepower (HP)	Safety Score (out of 100)	Brand
Car 1	120	95	Audi
Car 2	180	85	Audi
Car 3	240	70	Audi
Car 4	150	100	Ford
Car 5	200	95	Ford

Consider Table I as an example. Each car is associated with two attributes: horsepower and safety score. Suppose that Alice wants to buy a car. The system interacts with Alice through several pairwise comparisons, e.g., (1) comparing Car 1 vs. Car 2 and (2) comparing Car 3 vs. Car 4. If Alice consistently prefers a safer car, i.e., choosing Car 1 in the first comparison and Car 4 in the second comparison, we can gradually learn that safety matters more in Alice's preference. Guided by this information, the safest car will be returned to Alice.

Although promising, existing studies on the interactive query mainly focus on the *single-dataset* setting, i.e., all tuples are placed in a single dataset and the goal is to identify the user's global favorite tuples from this dataset. However, such a setting may not fully capture the complexities of real-world scenarios. In many real-world scenarios, tuples are naturally organized into multiple groups [17], [18], [19], [20]. These groups may reflect diversity requirements (e.g., cars from diverse brands), fairness constraints (e.g., people from different demographic groups), independent choice spaces (e.g., candidates from heterogeneous data sources). Worse still, tuples from different groups may not be directly comparable, and thus, users may want to identify the local best tuples from each group. Blindly merging these groups into a single dataset and applying existing single-dataset algorithms to identify global favorite tuples may cause the results to concentrate in only a few groups, leaving others uncovered. This not only reduces result diversity but may also raise serious fairness or discrimination concerns for minority or underrepresented groups.

Continuing with the car example (Table I), Alice may want to retrieve one car from each brand for further comparison. Although cars from different brands can be evaluated by the same set of criteria, i.e., horsepower and safety, each brand forms an independent set for diversity. If all cars are merged into a single dataset, querying the top-ranked cars from the merged dataset may return only cars from a single brand, failing to provide a diverse shortlist. Consider another real scenario of scholarship applications [21], where applicants are divided into diverse demographic groups based on race, ethnicity, gender, religion, and nationality, etc. The committee may invite shortlisted applicants for a panel interview. To pro-

vide an opportunity for members of underrepresented groups, strong candidates must be identified from each group. If these groups are merged and only the global favorites are selected, the finalists may consist of persons with the same (or majority) race, gender, or nationality. This raises severe fairness and discrimination concerns. Similar issues also arise in, e.g., job hiring, student admission or loan applications.

Motivated by this, we study a novel problem *Interactive Query over Multiple Datasets (IMD)*. Given multiple datasets for the same query task, the goal is to interactively ask the user questions to identify the user's local favorite tuple within each individual dataset, rather than simply returning the global favorite tuples from their union. As commonly practiced in interactive queries, the number of questions asked should be minimized, since excessive interactions may lead to user fatigue and degrade the user experience. To the best of our knowledge, we are the first to formulate and study the IMD problem.

To solve this problem, a straightforward idea is to sequentially apply existing single-dataset algorithms to each dataset. However, this suffers from substantial interaction overhead, since we ask questions independently from each dataset, some of which may be redundant and unnecessary. When the number of datasets increases, the total number of questions can place a significant burden on users and diminish their experience.

Therefore, we explore information sharing across multiple datasets. Note that although the favorite tuple must be identified within each dataset, all datasets are queried for the same user and the same query task. Thus, the user preference can be governed in a common decision space, allowing information learned from one dataset to be reused across the others. For example, the same user typically evaluates cars to purchase from different brands using the same set of attributes (e.g., horse power and safety); similarly, we generally apply a common set of criteria across different groups of people, when making hiring, admission and scholarship disbursement decisions for the *same* position. Following this idea, we develop two lines of algorithms with different levels of information sharing.

For the first line of algorithms, we propose a *shared utility range* (SUR) mechanism to propagate information across datasets. Instead of restarting interaction from scratch for every dataset, we maintain a global *utility range* that records the feasible range of $\boldsymbol{u}$ induced by the constraints from earlier interactions. When processing a new dataset, this shared range can be directly used to filter tuples that cannot be the local favorite in this dataset. In other words, the constraints collected from earlier datasets can be shared to later datasets, reducing redundant questions. Equipping existing algorithms [15], [22] with this mechanism, they are extended to several multi-dataset variants (SUR-HD-PI, SUR-RH, SUR-UH-RANDOM, and SUR-UH-SIMPLEX). However, these variants follow the sequential paradigm: questions are asked for one dataset at a time, with the utility range updated before moving to the next dataset.

For the second line of algorithms, we introduce a *shared questions* mechanism to break the sequential paradigm, by asking questions that are informative for multiple datasets at once, consolidating several dataset-specific interactions into a single shared interaction. Nevertheless, selecting such questions is challenging because datasets may vary in size and distribution. A question that is highly informative for one dataset may be useless for another. Therefore, we need to find good questions that benefit as many datasets as possible. To this end, we formalize the shared question selection problem, based on which we propose an effective heuristic for question selection.

However, when the number of datasets or tuples grows large, even the heuristic faces two computational bottlenecks. On the one hand, we must maintain up-to-date information for all datasets, which can be expensive; on the other hand, based on such information, we need to evaluate many candidate questions over multiple datasets, which can incur high latency. To address these bottlenecks, we adopt a localized strategy, which first identifies a small set of promising candidate tuples from each dataset based on the current information and then selects questions only within this localized set. This restricted selection substantially reduces the cost while keeping the number of questions small. Combining the shared questions mechanism and the localized strategy, we propose algorithm SHAREDQ.

Contributions. Our main contributions are as follows:

- To the best of our knowledge, we are the first to propose the *Interactive Query over Multiple Datasets (IMD)* problem, which aims to interactively find a user's favorite tuple in each given dataset. We also show a lower bound $\Omega(\log_2 N)$ on the number of questions asked for the IMD problem, where N is the maximum size of all datasets.
- We augment existing single-dataset algorithms with a *shared utility range* mechanism to support multi-dataset queries, yielding four variants (SUR-HD-PI, SUR-RH, SUR-UH-RANDOM, and SUR-UH-SIMPLEX). They reduce questions by propagating learned information across datasets.
- We develop SHAREDQ, an advanced algorithm designed for multi-dataset scenarios. It integrates a shared questions mechanism with a localized strategy to (a) optimize question selection over multiple datasets simultaneously and (b) take advantage of localization for reducing the cost.
- Extensive experiments on synthetic and real-world datasets demonstrate the effectiveness and efficiency of our proposed algorithms. Our algorithm SHAREDQ, compared to existing baselines, is able to reduce the number of questions asked by 50%–90% under typical settings.

The rest of the paper is structured as follows. We discuss related work in Section II and define the problem in Section III. After reviewing existing single-dataset baselines in Section IV, we present our algorithms with the shared utility range mechanism and the shared question mechanism in Sections V, and VI, respectively. Experimental results are presented in Section VII, while Section VIII concludes this paper.

II. RELATED WORK

Several existing studies introduce user interaction on the *multi-criteria decision-making* problem over a single dataset.

The interactive similarity query seeks to retrieve tuples that are close to a user's latent query tuple by iteratively learning both the latent query and the distance function [23], [24],

[25]. Intuitively, it presents many candidate tuples to the user and asks for their relevance scores. Based on the user feedback, they update the estimated query tuple or distance function accordingly. Although this paradigm is flexible, it typically relies on extensive explicit feedback, requiring users to examine and score a large number of tuples. Such a heavy interaction burden makes them difficult to apply in practical scenarios.

To reduce the interaction burden, the interactive regret-minimizing query replaces fine-grained relevance scoring with simpler preference comparison [26]. It returns a set of tuples whose *regret ratio*, a criterion for evaluating tuples, is below a specified threshold. In each interactive round, it presents several tuples, and the user only needs to select the most preferred one. According to the feedback, it shrinks the feasible region of the user's preference. When this region is small enough, the desired tuples can be returned. However, it displays *fake tuples* to the user during interaction, which are artificially constructed rather than selected from the database. To address this issue, [22] propose the *strongly truthful* regret-minimizing interactive query, whose displayed tuples are from the database.

Wang et al. [15] propose to return one of the top-k tuples by interacting with the user. They partition the preference space into equivalence regions, where all preferences in the same region correspond to the same representative tuples. The interaction process then uses the user's feedback to progressively eliminate regions that are inconsistent with the observed feedback. Once the target region is identified, the corresponding tuple is returned. More recent works [27], [28] explore the integration of different types of attributes into the interaction.

In the domain of machine learning, our problem is closely related to *learning to rank* [29], [30], [31], [32], [33], [34]. It aims to infer the ranking of tuples through user interaction. However, most existing algorithms [30], [31], [32] rely solely on (pairwise) comparison (i.e., tuple $\boldsymbol{p}$ is preferred over tuple $\boldsymbol{q}$) and do not utilize the underlying attribute semantics well. [29] accounts for such semantic, but it focuses on deriving the full ranking; this often involves redundant questions that contribute little to our goal of identifying the user's favorite tuples.

As discussed in Section I, all above algorithms primarily target a single dataset, assuming that users only want globally favorite tuples. This assumption may not hold in practice, since we often need to identify the local favorite tuples from multiple datasets, to promote diversity and fairness or mitigate discrimination. Simply applying existing single-dataset algorithms to each dataset is also inefficient due to the interaction overhead. To bridge this gap, we formulate the IMD problem and develop algorithms for interactive querying over multiple datasets.

III. Problem Definition

Dataset. The input to our problem is a collection of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$. Recent work [35] suggests that users typically focus on only a small number of decision-relevant attributes, e.g., fewer than five, when making queries. Thus, we do not require the complete schema of all datasets to be identical. It is enough if their query-relevant attributes can be represented in a common decision space. For simplicity, we assume w.l.o.g. that the tuples $\boldsymbol{p}$ from each dataset $\mathcal{D} \in \mathrm{D}$ are described by the same set of d query-relevant attributes, where the i-th attribute value of $\boldsymbol{p}$ is denoted as $p[i]$. Indeed, our methodology also accommodates some datasets missing certain decision-relevant attributes; we will explain how later.

Table II: Dataset

$\boldsymbol{p}$	$p[1]$	$p[2]$	$f_{\boldsymbol{u}}(\boldsymbol{p})$
$\boldsymbol{p_1}$	0.1	0.5	0.34
$\boldsymbol{p_2}$	0.3	0.4	0.36
$\boldsymbol{p_3}$	0.3	0.3	0.30
$\boldsymbol{p_4}$	0.5	0.2	0.32
$\boldsymbol{p_5}$	0.6	0.1	0.30

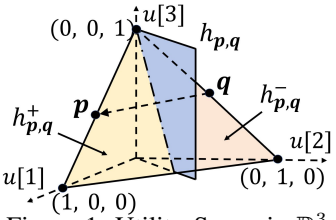


Figure 1: Utility Space in $\mathbb{R}^3$

Following existing studies [22], [15], [14], we regard each tuple as a point in a d-dimensional space $\mathbb{R}^d$. We use the terms *tuple/point* and *attribute/dimension* interchangeably in the rest of the paper. We assume w.l.o.g. that each dimension is normalized into the range $[0, 1]$ using min-max normalization.

User Preference. Following existing studies [22], [15], [14], [36], [37], we model the preference of each user using a *linear function*, denoted by $f_{\boldsymbol{u}}(\cdot)$, called *utility function*:

$$f_{\boldsymbol{u}}(\boldsymbol{p}) = \boldsymbol{u} \cdot \boldsymbol{p} = \textstyle\sum_{i=1}^{d} u[i]p[i].$$

It is parameterized by a non-negative vector $\boldsymbol{u} = (u[1], u[2], \ldots, u[d])$, namely the *utility vector*, which captures the user's preference on each attribute. A larger $u[i]$ means that the i-th attribute is more important to the user. We assume w.l.o.g. $\sum_{i=1}^{d} u[i] = 1$. The score $f_{\boldsymbol{u}}(\boldsymbol{p})$ is called the *utility* of $\boldsymbol{p}$. A higher utility indicates $\boldsymbol{p}$ is more preferred. For each $\mathcal{D}_i$, the user's favorite point is the one with the highest utility, i.e., $\boldsymbol{p}_i^* = \arg\max_{\boldsymbol{p} \in \mathcal{D}_i} f_{\boldsymbol{u}^*}(\boldsymbol{p})$, where $\boldsymbol{u}^*$ denotes the user's utility vector. The frequently used notations are shown in Table V.

As commonly practiced in the literature, we assume that a user has a unified preference across datasets during interaction. If preferences differ across datasets, there is little room for information sharing and the problem reduces to applying a single-dataset algorithm independently to each dataset.

Example 1. *Consider the points in Table II. Assume that the utility function is* $f_{\boldsymbol{u}}(\boldsymbol{p}) = 0.4p[1] + 0.6p[2]$*, i.e.,* $\boldsymbol{u} = (0.4, 0.6)$*. Then the utility of point* $\boldsymbol{p}_2$ *w.r.t.* $\boldsymbol{u}$ *is calculated as* $f_{\boldsymbol{u}}(\boldsymbol{p}_2) = 0.4 \times 0.3 + 0.6 \times 0.4 = 0.36$*. The utilities of other candidate points can be computed similarly.*

Problem IMD. If the user's utility vector $\boldsymbol{u}^*$ is known, we can directly identify the favorite point in each dataset. However, users often have difficulty specifying their preferences precisely [26], [22]. Therefore, we adapt the iterative framework in [22], [15], [14], which consists of three phases in each iterative round: (1) *Point Selection*, which selects a pair of points as the question for user comparison; (2) *Information Maintenance*, which updates the maintained information based on user feedback; and (3) *Stopping Condition*, which terminates the interaction once the favorite point in every dataset can be identified. Otherwise, the process continues to the next round.

Formally, we are interested in the following problem.

Problem 1 (*Interactive Query over Multiple Datasets (IMD)*). *Given a set of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$, the goal is to interactively identify the user's favorite point within each dataset $\mathcal{D}_i$, minimizing the total number of questions asked.*

The following theorem presents a lower bound on the number of questions asked for problem IMD. Due to the lack of space, the proofs of theorems/lemmas can be found in [38].

Theorem 1. *Given m datasets with $N = \max_{i\in[1,m]} |\mathcal{D}_i|$, any pairwise-comparison-based interactive algorithm requires $\Omega(\log_2 N)$ rounds to identify the favorite point in each dataset.*

To the best of our knowledge, we are the first to study the IMD problem. Its central challenge is to adapt all three phases of the interactive framework to the multi-dataset setting. Existing interactive algorithms are designed for a single dataset: they select questions, maintain information, and determine when to stop with respect to that dedicated dataset. These do not work well for problem IMD where (a) questions should be informative across multiple datasets, (b) feedback information should be shared among them, and (c) the stopping condition must ensure that the favorite point has been identified in every dataset. Therefore, novel designs are required for all phases.

Remark. Recall that for simplicity, we assume that the tuples from each dataset $\mathcal{D}_i \in \mathrm{D}$ are described by the same set of decision-relevant attributes. In practice, however, it is possible that some attributes are missing from a dataset $\mathcal{D}_i$. As can be seen later, if this is the case, we can simply set the values of the missing attribute for all tuples in $\mathcal{D}_i$ to be 0, while keeping the rest of our method intact, since we only compare two tuples from the same dataset in each interaction. To some extent, this also motivates the multi-dataset setting: when certain attributes are missing in one dataset, tuples from this dataset may not be directly comparable to those from other datasets and thus, it is better to identify the best from each dataset and present them as a shortlist to the user for final decision-making.

IV. Baseline Adaptation

In this section, we first introduce necessary preliminaries. Then, we revisit existing algorithms UH-Random [22], UH-Simplex [22], RH [15], and HD-PI [15] and show how to adapt them to solve our problem IMD.

A. Preliminaries

Existing works study the interactive query from a geometric perspective. Below are some relevant concepts.

Utility Space. Since the utility vector $\boldsymbol{u}$ is non-negative and normalized, its domain forms a $(d-1)$-dimensional polytope, called the *utility space* and denoted as $\mathcal{U} = \{\boldsymbol{u} \in \mathbb{R}^d_+ \mid \sum_{i=1}^d u[i] = 1\}$. For instance, when $d = 3$ in Figure 1, the utility space $\mathcal{U}$ corresponds to a triangle in space $\mathbb{R}^3_+$.

Hyper-planes and Half-spaces. For any pair of points $\boldsymbol{p}, \boldsymbol{q} \in \mathrm{D}$ (see Figure 1), they define a hyper-plane in space $\mathbb{R}^d_+$:

$$h_{\boldsymbol{p},\boldsymbol{q}} = \{\boldsymbol{u} \in \mathbb{R}^d_+ \mid \boldsymbol{u} \cdot (\boldsymbol{p} - \boldsymbol{q}) = 0\}.$$

Table III: Frequently Used Notations

Notation	Definition
D and $\mathcal{D}_i$	All datasets and the i-th dataset.
m and n_i	Number of datasets and points in $\mathcal{D}_i$.
$\boldsymbol{p}$ and d	A point and dimensionality.
$\boldsymbol{u}$ and $\mathcal{R}$	Utility vector and global utility range.
$\Theta_{\boldsymbol{p}}$	Partition of point $\boldsymbol{p}$.
$h_{\boldsymbol{p},\boldsymbol{q}}$	Hyper-plane of $\boldsymbol{p}$ and $\boldsymbol{q}$.
$h^+_{\boldsymbol{p},\boldsymbol{q}}/h^-_{\boldsymbol{p},\boldsymbol{q}}$	Positive/negative half-space of $h_{\boldsymbol{p},\boldsymbol{q}}$.

This hyper-plane divides $\mathcal{U}$ into two half-spaces, namely $h^+_{\boldsymbol{p},\boldsymbol{q}}$ and $h^-_{\boldsymbol{p},\boldsymbol{q}}$. The *positive half-space* $h^+_{\boldsymbol{p},\boldsymbol{q}}$ encompasses all utility vectors where $\boldsymbol{p}$ is preferred to $\boldsymbol{q}$ (i.e., $\boldsymbol{u}\cdot\boldsymbol{p} > \boldsymbol{u}\cdot\boldsymbol{q}$, denoted by $\boldsymbol{p} \succ \boldsymbol{q}$), whereas the *negative half-space* $h^-_{\boldsymbol{p},\boldsymbol{q}}$ contains those where $\boldsymbol{q}$ is preferred to $\boldsymbol{p}$ (i.e., $\boldsymbol{u}\cdot\boldsymbol{p} < \boldsymbol{u}\cdot\boldsymbol{q}$, denoted by $\boldsymbol{p} \prec \boldsymbol{q}$).

Lemma 1 ([15], [27]). *If a user indicates $\boldsymbol{p} \succ \boldsymbol{q}$ (resp. $\boldsymbol{p} \prec \boldsymbol{q}$), the user's utility vector must lie in $h^+_{\boldsymbol{p},\boldsymbol{q}}$ (resp. $h^-_{\boldsymbol{p},\boldsymbol{q}}$).*

B. Algorithms

Existing algorithms, including UH-Random [22], UH-Simplex [22], RH [15], and HD-PI [15], target the single-dataset setting. Despite their different strategies, their underlying mechanisms can be abstracted into a unified framework. The key is to learn the user's utility vector based on the user feedback, by interpreting each feedback as a constraint, i.e., $h^+_{\boldsymbol{p},\boldsymbol{q}}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}}$, in $\mathcal{U}$ to restrict the possible domain of $\boldsymbol{u}^*$.

They maintain two structures for this purpose: (1) a utility range $\mathcal{R} \subseteq \mathcal{U}$ containing all utility vectors consistent with the user's feedback (an example of $\mathcal{R}$ is given in the orange region in Figure 2), and (2) a candidate set $\mathcal{C}$ containing all points that may be the user's favorite. Initially, $\mathcal{R} = \mathcal{U}$ and $\mathcal{C} = \mathcal{D}$. In each round, the algorithms select a pair of points $\langle \boldsymbol{p}, \boldsymbol{q} \rangle$ as the question for pairwise comparison. According to the user feedback, they update $\mathcal{R}$ to either $h^+_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$ following Lemma 1 and remove points from $\mathcal{C}$ that cannot be one with the highest utility under *any* utility vector in $\mathcal{R}$. The interaction stops when $|\mathcal{C}| = 1$ and the point in $\mathcal{C}$ is returned.

Existing algorithms differ mainly in question selection. UH-Random and UH-Simplex select questions from potentially optimal points [22]. RH favors the pairs of points whose hyper-planes are close to the center of $\mathcal{R}$ [15] and HD-PI selects questions according to the distribution of $\mathcal{C}$ [15].

To establish the baselines for problem IMD, we can apply an existing algorithm to each dataset in sequence, independently identifying the user's favorite point within that dataset. The process terminates once this is done for all datasets.

The following theorem gives the upper bound on the number of questions asked for the adaptation of algorithm UH-Simplex. The results of other baseline adaptations are similar.

Theorem 2. *Given a collection of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$, where each dataset $\mathcal{D}_i$ consists of n_i points, the adaptation of algorithm* UH-Simplex *can solve problem IMD in $O\left(\sum_{i=1}^m n_i\right)$ interactive rounds in the worst case, and $O(deg_{max} \cdot \sum_{i=1}^m \sqrt[d]{n_i})$ on average, where deg_{max} is a parameter for evaluating the distribution of the datasets.*

V. Algorithms based on Shared Utility Range

Although sequentially applying existing algorithms over datasets is straightforward, it incurs substantial interaction overhead since it treats datasets independently and discards preference information learned from earlier ones. Since all datasets are queried by the same user for the same task, the underlying utility vector remains fixed across datasets. This suggests that preference information learned from one dataset can inform queries on others. Specifically, each user feedback induces a half-space that constrains the utility range; since the user's utility vector is fixed across all datasets, the constraints remain valid across datasets. Formally, we have Lemma 2.

Lemma 2. *If the user feedback from previously processed datasets induces a utility range $\mathcal{R}$ formed by intersecting the half-spaces, the user's utility vector lies in $\mathcal{R}$ and $\mathcal{R}$ safely serves as the initial utility range for any unprocessed dataset.*

Following Lemma 2, we introduce a *shared utility range (*SUR*)* mechanism to enhance existing algorithms. The main idea is to maintain a global utility range $\mathcal{R}$ throughout the entire interaction process. When we finish processing the $(i-1)$-th dataset $\mathcal{D}_{i-1}$, the resulting utility range $\mathcal{R}_{i-1}$ is propagated to the next dataset $\mathcal{D}_i$ as its initial utility range $\mathcal{R}_i$, instead of restarting the interaction from the entire utility space $\mathcal{U}$. This allows subsequent datasets to start from a smaller range, avoiding questions that learn useless preference constraints.

Building upon the utility range $\mathcal{R}_{i-1}$ obtained from the $(i-1)$-th dataset $\mathcal{D}_{i-1}$, we can not only initialize $\mathcal{R}_i$, but also improve the initialization of the candidate set $\mathcal{C}_i$ for the next dataset $\mathcal{D}_i$. Instead of initializing $\mathcal{C}_i$ as the entire dataset $\mathcal{D}_i$, we retain only those points that might be the user's favorite one. Specifically, since the user's utility vector is guaranteed to reside within $\mathcal{R}_{i-1}$, a point $\boldsymbol{p} \in \mathcal{D}_i$ can be safely pruned if no utility vector $\boldsymbol{u} \in \mathcal{R}_{i-1}$ makes $\boldsymbol{p}$ a highest-utility point in $\mathcal{D}_i$. The following lemma presents a sufficient condition to determine whether a point can be pruned.

Lemma 3. *If there exists a point $\boldsymbol{q} \in \mathcal{D}_i$ such that $\mathcal{R}_{i-1} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$, then for any $\boldsymbol{u} \in \mathcal{R}_{i-1}$, $\boldsymbol{p}$ is not the highest-utility point.*

To efficiently verify whether $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$, we utilize the *extreme points* of $\mathcal{R}$ [39]. Here, the extreme points are the corner points of a polytope. For example, in Figure 1, (1, 0, 0), (0, 1, 0), and (0, 0, 1) are the extreme points of the utility space.

Lemma 4. *If all extreme points of $\mathcal{R}$ lie in $h^+_{\boldsymbol{q},\boldsymbol{p}}$, $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$.*

To maximize the pruning efficiency, we introduce a *hard-to-easy* heuristic by pre-sorting the datasets in D in descending order based on their sizes. The intuition is that larger datasets contain more points, and thus, induce a larger pool of candidate interactive questions. This gives more opportunities to select informative questions that can effectively shrink $\mathcal{R}$. Once a tighter global range is obtained, it can be propagated to the remaining datasets and benefit their candidate set pruning.

With the shared utility range mechanism, we enhance existing algorithms: UH-Random [22], UH-Simplex [22], RH [15], and HD-PI [15]. The pseudocode of the enhanced ones (SUR-UH-Random, SUR-UH-Simplex, SUR-RH, and SUR-HD-PI) is shown in Algorithm 1. In the beginning, we initialize the global utility range $\mathcal{R}_0$ as the entire utility space $\mathcal{U}$ and the output set S to be empty (line 3). Then, the datasets are sorted in descending order based on their sizes (line 4). After sorting, the datasets are processed sequentially. Consider the i-th dataset $\mathcal{D}_i$. We create a candidate set $\mathcal{C}_i$ initialized to be $\mathcal{D}_i$ and refine it by pruning points based on Lemma 3 and Lemma 4 (line 6). After obtaining $\mathcal{C}_i$, we apply existing algorithms on $\mathcal{C}_i$ with $\mathcal{R}_{i-1}$ as its initial utility range (line 7). Once the user's favorite point within $\mathcal{D}_i$ is identified, it is added to S, and the refined utility range returned by the existing algorithm is retained as the global utility range for subsequent datasets (line 8). This process continues until all datasets have been processed, after which S is returned (line 9).

Algorithm 1: SUR-ExistingAlg

```
1 Input: m datasets D = {D_1, D_2, ..., D_m};
2 Output: The set of the user's favorite points S;
3 Initialize the global utility range R_0 ← U and set S ← ∅ ;
4 Sort D by the size of each D_i in descending order;
5 for i = 1 to m do
6     C_i ← D_i, and prune invalid points in C_i based on R_{i-1};
7     p_i^*, R_i ← ExistingAlg(C_i, R_{i-1});
8     S ← S ∪ {p_i^*};
9 return S;
```

Take the existing algorithm UH-Simplex as an example. We analyze its performance as follows. Let $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$ be a set of datasets, sorted in descending order based on their sizes such that $n_1 \geq n_2 \geq \cdots \geq n_m$. Let $N = n_1$ be the maximum dataset size. Let $V(\mathcal{R}_{i-1})$ and $V(\mathcal{U})$ denote the volume of $\mathcal{R}_{i-1}$ and $\mathcal{U}$, respectively.

Theorem 3. *Suppose that all points are uniformly distributed on a spherical surface,* SUR-UH-Simplex *finds the user's favorite point in each dataset using $O(\sum_{i=1}^{m} n_{\mathcal{R}_{i-1}})$ interactive rounds in the worst case, and $O\left(deg_{max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_{\mathcal{R}_{i-1}}}\right)$ rounds on average, where $n_{\mathcal{R}_{i-1}} = \lceil n_i \cdot \frac{V(\mathcal{R}_{i-1})}{V(\mathcal{U})} \rceil$ and deg_{max} is a parameter used to evaluate the distribution of the datasets.*

VI. Algorithms based on Shared Questions

In this section, we present our algorithm SharedQ. We first introduce the holistic idea of selecting questions across the global spectrum of datasets. Then, we propose optimization strategies to minimize the underlying computational burden.

A. Hyper-plane Selection Across Datasets

Although SUR-ExistingAlg share a global utility range during the interaction, they still sequentially process datasets one by one and select questions based on each *single* dataset. However, each selection may potentially influence *multiple* datasets. Specifically, when a question compares $\boldsymbol{p}$ and $\boldsymbol{q}$, the user's response induces a half-space, e.g., $h^+_{\boldsymbol{p},\boldsymbol{q}}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}}$, refines the global utility range $\mathcal{R}$. When they proceed to another dataset, this refined range is used to initialize its candidate set

through point pruning. Thus, a question selected based on one dataset can enable pruning across multiple datasets. In light of this, we propose a mechanism termed *shared questions*, which selects questions jointly across multiple datasets so that the resulting refinement of global $\mathcal{R}$ can benefit them all.

Nevertheless, selecting a question by considering multiple datasets is non-trivial. There are two issues that need to be addressed: (1) how to measure whether a question is informative, and (2) how to measure the question across multiple datasets.

For the first issue, we focus on the output of problem IMD. The informativeness of a question is measured by how effectively it helps rule out candidate points that could be the output. To capture this, we use the notion of *partition*.

Definition 1 (Partition)**.** *For each $\boldsymbol{p} \in \mathcal{D}_i$, we delineate a unique range in $\mathcal{U}$, called partition and denoted by $\Theta_{\boldsymbol{p}}$. It contains all $\boldsymbol{u} \in \mathcal{U}$ under which $\boldsymbol{p}$ has the highest utility among $\mathcal{D}_i$, i.e., $\Theta_{\boldsymbol{p}} = \{\boldsymbol{u} \in \mathcal{U} \mid \forall \boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\},\ f_{\boldsymbol{u}}(\boldsymbol{p}) > f_{\boldsymbol{u}}(\boldsymbol{q})\}$.*

To build the partition for a point $\boldsymbol{p} \in \mathcal{D}_i$, we utilize half-spaces. For each $\boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\}$, we build a half-space $h^+_{\boldsymbol{p},\boldsymbol{q}}$. The following lemma ensures that the intersection of these half-spaces with $\mathcal{U}$ forms the partition, i.e.,

$$\Theta_{\boldsymbol{p}} = \left(\bigcap_{\boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\}} h^+_{\boldsymbol{p},\boldsymbol{q}} \right) \cap \mathcal{U}.$$

Lemma 5. *The intersection of all $h^+_{\boldsymbol{p},\boldsymbol{q}}$ contains all utility vectors $\boldsymbol{u} \in \mathcal{U}$ under which $\boldsymbol{p}$ has the highest utility among $\mathcal{D}_i$.*

Example 2. *As shown in Figure 2, the partition $\Theta_{\boldsymbol{p}}$ of $\boldsymbol{p}$ is shown in the blue region, which is bounded by three half-spaces, namely $h^+_{\boldsymbol{p},\boldsymbol{q}}$, $h^+_{\boldsymbol{p},\boldsymbol{s}}$ and $h^+_{\boldsymbol{p},\boldsymbol{v}}$. By Lemma 5, for every utility vector in $\Theta_{\boldsymbol{p}}$, $\boldsymbol{p}$ has a higher utility than $\boldsymbol{q}$, $\boldsymbol{s}$ and $\boldsymbol{v}$.*

For each point $\boldsymbol{p} \in \mathcal{D}_i$, if the intersection of $\Theta_{\boldsymbol{p}}$ and $\mathcal{R}$ is empty, i.e., $\Theta_{\boldsymbol{p}} \cap \mathcal{R} = \emptyset$, there does not exist any utility vector $\boldsymbol{u} \in \mathcal{R}$ under which $\boldsymbol{p}$ has the highest utility among $\mathcal{D}_i$. Since the user's utility vector $\boldsymbol{u}^*$ is guaranteed to lie in $\mathcal{R}$, point $\boldsymbol{p}$ cannot have the highest utility w.r.t. $\boldsymbol{u}^*$ among $\mathcal{D}_i$. This means that $\boldsymbol{p}$ can be safely ruled out.

Consider a question with points $\boldsymbol{p}$ and $\boldsymbol{q}$, which induces the hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$. Depending on the user feedback, the utility range $\mathcal{R}$ will be refined to either $h^+_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$. This refinement may make some partitions disjoint from updated $\mathcal{R}$ (i.e., $\Theta_{\boldsymbol{p}} \cap \mathcal{R} = \emptyset$), so their corresponding points can no longer be the user's favorite and can be safely pruned. Following this observation, the informativeness of a question can be measured by the number of partitions it eliminates based on $\mathcal{R}$.

However, the user's response is unknown before the question is asked, so partitions may be eliminated from either $h^+_{\boldsymbol{p},\boldsymbol{q}}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}}$. A question that eliminates many partitions when the user prefers $\boldsymbol{p}$ to $\boldsymbol{q}$ may eliminate few when the user prefers $\boldsymbol{q}$ to $\boldsymbol{p}$. To ensure informativeness regardless of the response, we evaluate both possible outcomes and use the worst-case to estimate its effect. Specifically, with a slight abuse of notations, we use $|h^+_{\boldsymbol{p},\boldsymbol{q}}|$ and $|h^-_{\boldsymbol{p},\boldsymbol{q}}|$ to denote the numbers of partitions in $h^+_{\boldsymbol{p},\boldsymbol{q}}$ and $h^-_{\boldsymbol{p},\boldsymbol{q}}$, respectively. The elimination effect of $h_{\boldsymbol{p},\boldsymbol{q}}$ on $\mathcal{D}_i$ is measured by $E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_i) = \min\{|h^+_{\boldsymbol{p},\boldsymbol{q}}|, |h^-_{\boldsymbol{p},\boldsymbol{q}}|\}$. This gives a lower bound on the number of partitions eliminated, regardless of the user's response.

Example 3. *In Figure 3, we show a hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$ from $\mathcal{D}_1$, where the arrow points to its positive half-space. Since $\Theta_{\boldsymbol{p}_1} \subseteq h^+_{\boldsymbol{p},\boldsymbol{q}}$ and $\Theta_{\boldsymbol{p}_2}, \Theta_{\boldsymbol{p}_3}, \Theta_{\boldsymbol{p}_4} \subseteq h^-_{\boldsymbol{p},\boldsymbol{q}}$, we have $E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_1) = \min(|h^+_{\boldsymbol{p},\boldsymbol{q}}|, |h^-_{\boldsymbol{p},\boldsymbol{q}}|) = \min(1, 3) = 1$.*

For the second issue, we aggregate the elimination effects over all datasets. A question may eliminate many candidates in one dataset but few in another. Since datasets may differ in size, and thus in the number of partitions, we weight each dataset according to the size of its current candidate set $\mathcal{C}_i$: $W_i = |\mathcal{C}_i| / (\sum_{j=1}^{m} |\mathcal{C}_j|)$. Datasets with more candidates receive greater weight, since they generally require more effort to narrow down to a single point. Then, we define the *pruning score*.

Definition 2 (Pruning Score)**.** *Given $h_{\boldsymbol{p},\boldsymbol{q}}$, its pruning score is $s(h_{\boldsymbol{p},\boldsymbol{q}}) = \sum_{i=1}^{m} W_i \times E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_i)$, where $E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_i) = \min\{|h^+_{\boldsymbol{p},\boldsymbol{q}}|, |h^-_{\boldsymbol{p},\boldsymbol{q}}|\}$ and $W_i = |\mathcal{C}_i| / (\sum_{j=1}^{m} |\mathcal{C}_j|)$.*

The pruning score connects multiple datasets together: each dataset contributes based on (1) the number of remaining points and (2) the number of points the question can eliminate. We select the hyper-plane with the highest pruning score as the next question. This favors questions that offer pruning benefits across multiple datasets, rather than being useful for just one.

The scoring function above formulates the question selection as a global optimization problem. Unfortunately, its computation is prohibitively expensive. The cost arises from two factors: the number of hyper-planes to evaluate and the number of partitions to examine for each hyper-plane. Consider the m datasets, where each one $\mathcal{D}_i$ contains n_i points. Since each pair of points may induce a candidate hyper-plane, the total number of hyper-planes is $O(\sum_{i=1}^{m} n_i^2)$. Moreover, each point may correspond to one output partition, so evaluating a hyper-plane requires checking its pruning effect over up to $O(\sum_{i=1}^{m} n_i)$ partitions across all datasets. To address this issue, we develop two acceleration strategies: the first reduces the number of hyper-planes considered, while the second reduces the number of points examined during the scoring process.

B. Hyper-plane Reduction

Let $\mathcal{H}_i$ denote the set of hyper-planes induced by point pairs in dataset $\mathcal{D}_i$. To reduce the number of hyper-planes to be evaluated, we adopt a two-stage strategy. In the first stage, we select a representative hyper-plane h_i from the local pool $\mathcal{H}_i$ of each dataset $\mathcal{D}_i$. This representative hyper-plane is intended to split the partitions effectively, without requiring us to compute the pruning score for every hyper-plane in $\mathcal{H}_i$. In the second stage, we evaluate the pruning score only for the m representative hyper-planes $\{h_1, h_2, \ldots, h_m\}$ and choose the one with the largest score as the next question.

To select a representative hyper-plane from each dataset, we propose an approximation strategy based on PCA [40], [41]. The key idea is to first construct a reference hyper-plane that approximately divides the candidate partitions into two

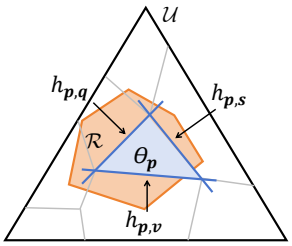


Figure 2: Partition and $\mathcal{R}$

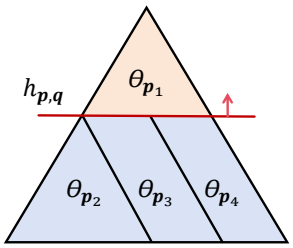


Figure 3: $E(h_{p,q}, \mathcal{D}_i)$

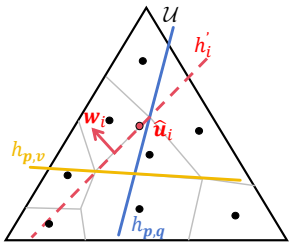


Figure 4: PCA Selection

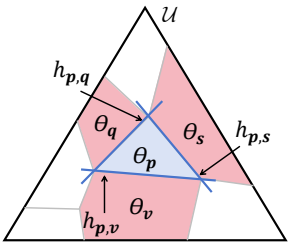


Figure 5: Find Neighbors

balanced groups, and then find a valid candidate hyper-plane most closely aligns with it. Specifically, for each point $\boldsymbol{p} \in \mathcal{C}_i$, we denote the geometric center of $\Theta_{\boldsymbol{p}} \cap \mathcal{R}$ by $\boldsymbol{u}_i(\boldsymbol{p})$. The set of centers $\{\boldsymbol{u}_i(\boldsymbol{p}) \mid \boldsymbol{p} \in \mathcal{C}_i\}$ summarizes the distribution of partitions for dataset $\mathcal{D}_i$. We then apply PCA to these centers and obtain the first principal direction $\boldsymbol{w}_i$ with $|\boldsymbol{w}_i| = 1$. This direction captures the largest spread of these centers $\boldsymbol{u}_i(\boldsymbol{p})$.

We then project each center $\boldsymbol{u}_i(\boldsymbol{p})$ onto $\boldsymbol{w}_i$, obtaining the projected point, i.e., $\big(\boldsymbol{w}_i \cdot \boldsymbol{u}_i(\boldsymbol{p})\big)\boldsymbol{w}_i$. We order these projected points along $\boldsymbol{w}_i$ and denote the median one by $\hat{\boldsymbol{u}}_i$. The reference hyper-plane is defined as $h_i' : \boldsymbol{w}_i \cdot (\boldsymbol{u} - \hat{\boldsymbol{u}}_i) = 0$. Intuitively, a center lies on either side of h_i' according to whether its projected point orders lower or higher than $\hat{\boldsymbol{u}}_i$. Thus, PCA selects a direction $\boldsymbol{w}_i$ along which centers have the largest variance, while the median ensures that the two sides contain approximately equal numbers of partition centers.

Note that hyper-plane h_i' is only an ideal reference and may not correspond to a valid question in $\mathcal{D}_i$, because valid questions must be selected from $\mathcal{H}_i$. Therefore, we select the representative hyper-plane in $\mathcal{H}_i$ that best approximates h_i'. For a hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$, we define its difference from h_i' as

$$\mathrm{diff}(h_{\boldsymbol{p},\boldsymbol{q}}, h_i') = \alpha \left(1 - \frac{|\boldsymbol{w}_i \cdot (\boldsymbol{p} - \boldsymbol{q})|}{\|\boldsymbol{p} - \boldsymbol{q}\|}\right) + \beta \frac{|\hat{\boldsymbol{u}}_i \cdot (\boldsymbol{p} - \boldsymbol{q})|}{\|\boldsymbol{p} - \boldsymbol{q}\|}$$

Here, the first term favors a hyper-plane better aligning with the reference direction, while the second encourages it to pass the projected median. The two parameters $\alpha, \beta > 0$ balance these two criteria. We choose the hyper-plane that minimizes this difference score, i.e., $h_i = \arg\min_{h \in \mathcal{H}_i} \mathrm{diff}(h, h_i')$.

Example 4. *In Figure 4, we first obtain the geometric center of each partition, shown in the black dots. By applying PCA and projection on these centers, we get the first principal direction $\boldsymbol{w}_i$ (the red arrow) and the median point $\hat{\boldsymbol{u}}_i$ (the red dot), yielding the reference hyper-plane h_i' (the red dashed line).*

Since h_i' does not correspond to a valid hyper-plane in $\mathcal{D}_i$, we seek a valid hyper-plane best approximate it. Consider two hyper-planes $h_{\boldsymbol{p},\boldsymbol{q}}$ (the blue line) and $h_{\boldsymbol{p},\boldsymbol{v}}$ (the yellow line). By definition, the diff *score of $h_{\boldsymbol{p},\boldsymbol{q}}$ will be lower because it aligns more closely with h_i' and passes near $\hat{\boldsymbol{u}}_i$.*

After obtaining one representative hyper-plane from each dataset, we compute the global score $s(h_i)$ for all $i \in 1, \ldots, m$ and select $h^* = \arg\max_{h_i \in \{h_1, \ldots, h_m\}} s(h_i)$. Thus, the PCA-based strategy reduces the number of hyper-planes requiring expensive global evaluation to only m representatives.

Theorem 4. *The PCA-based method reduces the time cost of question selection from $O\left(\left(\sum_{i=1}^m n_i^2\right) \cdot \left(\sum_{i=1}^m n_i\right) \cdot v\right)$ to $O\left(\sum_{i=1}^m n_i(d^2 + m \cdot v) + m \cdot d^3\right)$, where v is the maximum number of vertices for a partition.*

C. Point Reduction

We next reduce the number of points that needs to be considered in $\mathcal{C}_i$ for each dataset $\mathcal{D}_i$. Our key idea is to localize the computation. Instead of considering the entire utility range $\mathcal{R}$, we maintain a localized utility range $\mathcal{R}_l \subseteq \mathcal{R}$ and only consider points whose partitions intersect $\mathcal{R}_l$ during the scoring process. Since all subsequent questions are generated on these localized points, both the numbers of candidate partitions and candidate hyper-planes are significantly reduced.

Following this idea, we propose an effective strategy, called *Detect-and-Divide*. Intuitively, it maintains a localized utility range $\mathcal{R}_l \subseteq \mathcal{R}$ and iteratively interleaves a *detect* step and a *divide* step: (1) The *detect* step finds the points whose partitions intersect $\mathcal{R}_l$ and uses them to form localized candidate sets $\mathrm{L} = \{\mathcal{L}_1, \mathcal{L}_2, \ldots, \mathcal{L}_m\}$ where $\mathcal{L}_i \subseteq \mathcal{D}_i$ and hyper-plane pools $\mathrm{H} = \{\mathcal{H}_1, \mathcal{H}_2, \ldots, \mathcal{H}_m\}$, obtained from point pairs in each $\mathcal{L}_i$. (2) The *divide* step updates the utility range through user interactions until only one point remains in each localized candidate set $\mathcal{L}_i$. If for some $\mathcal{D}_i$, the remaining point in $\mathcal{L}_i$ is not the user's favorite point, the two steps are repeated with a refined localized range. This iterative process terminates only when the user's favorite points are found for all datasets. As can be noticed, these two steps replace global partition construction and hyper-plane enumeration with localized computation, substantially reducing the number of points examined.

Phase 1: Detect. The objective of the detect phase is to identify all the partitions that intersect $\mathcal{R}_l$ for each dataset $\mathcal{D}_i$. The points associated with these partitions form the localized candidate set $\mathcal{L}_i$. Candidate hyper-planes are then generated only from point pairs in $\mathcal{L}_i$, reducing the subsequent cost.

To achieve this, a naive approach is to construct the partition of every point in $\mathcal{D}_i$ and test if it intersects $\mathcal{R}_l$. However, this requires examining the entire dataset, and thus, largely defeats the purpose of localization. To avoid such a global scan, we exploit the adjacency structure of partitions to only build a few partitions. We call a hyper-plane that forms part of a partition's boundary a *bounding hyper-plane* of that partition. For example, as shown in Figure 5, $h_{\boldsymbol{p},\boldsymbol{q}}$, $h_{\boldsymbol{p},\boldsymbol{s}}$, and $h_{\boldsymbol{p},\boldsymbol{v}}$ are the

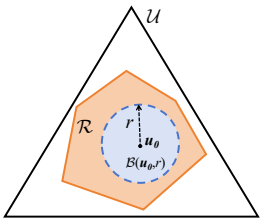


Figure 6: $\mathcal{B}(\boldsymbol{u}_0, r)$

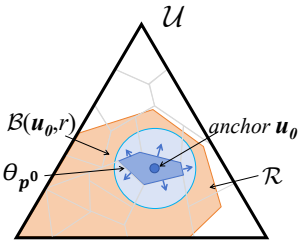


Figure 7: Detect: $\mathcal{R}_l$

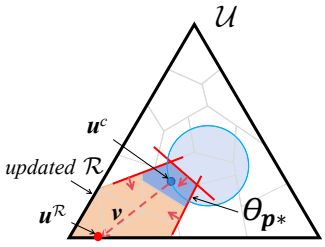


Figure 8: Ray Shooting

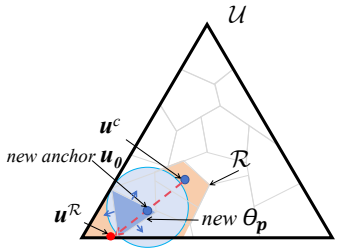


Figure 9: New Iteration

bounding hyper-planes of partition $\Theta_{\boldsymbol{p}}$. Based on these bounding hyper-planes, we define *neighboring partitions* as follows.

Definition 3 (Neighboring Partition)**.** *Two non-empty partitions $\Theta_{\boldsymbol{p}}$ and $\Theta_{\boldsymbol{q}}$ are neighboring partitions if they share a common bounding hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$. Correspondingly, their associated points $\boldsymbol{p}$ and $\boldsymbol{q}$ are called* p-neighbors*.*

We use the concept of *partition neighbors* to provide the basis for discovering partitions through local expansion.

Lemma 6. *If $\Theta_{\boldsymbol{p}} \neq \emptyset$ and $h_{\boldsymbol{p},\boldsymbol{q}}$ is a bounding hyper-plane of $\Theta_{\boldsymbol{p}}$, then $\Theta_{\boldsymbol{q}} \neq \emptyset$ and $\Theta_{\boldsymbol{q}}$ is adjacent to $\Theta_{\boldsymbol{p}}$ along $h_{\boldsymbol{p},\boldsymbol{q}}$.*

Example 5. *In Figure 5, the partition $\Theta_{\boldsymbol{p}}$ has three neighboring partitions, namely $\Theta_{\boldsymbol{q}}$, $\Theta_{\boldsymbol{s}}$, and $\Theta_{\boldsymbol{v}}$ (the red regions). By Lemma 6, these neighboring partitions can be identified by examining the corresponding bounding hyper-planes $h_{\boldsymbol{p},\boldsymbol{q}}$, $h_{\boldsymbol{p},\boldsymbol{s}}$, and $h_{\boldsymbol{p},\boldsymbol{v}}$, respectively (the blue lines).*

Lemma 6 implies that the neighbors of a partition can be identified from its bounding hyper-planes. Therefore, instead of constructing partitions all at once, we start from one known partition and recursively visit its neighboring partitions. Specifically, for each dataset $\mathcal{D}_i$, we first identify an anchor utility vector $\boldsymbol{u}_0$ in $\mathcal{R}_l$ (which is randomly sampled in the first iteration or selected strategically in the later iterations; we will discuss the strategy later). Then, we find the point with the highest utility w.r.t. $\boldsymbol{u}_0$, i.e., $\boldsymbol{p}_i^0 = \arg\max_{\boldsymbol{p}\in\mathcal{D}_i} f_{\boldsymbol{u}_0}(\boldsymbol{p})$. One can verify that $\Theta_{\boldsymbol{p}_i^0} \cap \mathcal{R}_l \neq \emptyset$ since $\boldsymbol{u}_0 \in \mathcal{R}_l$ and $\boldsymbol{u}_0 \in \Theta_{\boldsymbol{p}_i^0}$. We use $\boldsymbol{p}_i^0$ as the anchor point and initialize $\mathcal{L}_i = \{\boldsymbol{p}_i^0\}$.

Starting from this anchor point, we recursively explore its neighboring partitions. For each unvisited point $\boldsymbol{p} \in \mathcal{L}_i$, we construct its partition and identify its bounding hyper-planes. By Lemma 6, every bounding hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$ identifies a neighboring partition $\Theta_{\boldsymbol{q}}$. The corresponding $\boldsymbol{q}$ is added to $\mathcal{L}_i$ if its partition intersects $\mathcal{R}_l$. The bounding hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$ is inserted into $\mathcal{H}_i$ for subsequent question generation. This expansion continues until all the points in $\mathcal{L}_i$ have been visited.

Example 6. *In Figure 7, we identify the highest-utility point $\boldsymbol{p}^0$ w.r.t. an anchor utility vector $\boldsymbol{u}_0$ (the blue dot). We insert $\boldsymbol{p}^0$ into $\mathcal{L}_i$, construct its partition $\Theta_{\boldsymbol{p}^0}$ (the dark blue region) and examine its bounding hyper-planes. These bounding hyper-planes are inserted into $\mathcal{H}_i$ and the corresponding p-neighbors of $\boldsymbol{p}^0$ are appended to $\mathcal{L}_i$, which are processed recursively.*

It remains to show (1) how to define an appropriate sub-range $\mathcal{R}_l$, and (2) how to check if a partition intersects $\mathcal{R}_l$.

For the first issue, we define the sub-range based on a hyper-sphere for its flexibility. Given the anchor utility vector $\boldsymbol{u}_0$ selected above and a radius r, we denote by $\mathcal{B}(\boldsymbol{u}_0, r)$ the hyper-sphere centered at $\boldsymbol{u}_0$ with radius r (see Figure 6). We define $\mathcal{R}_l = \mathcal{R} \cap \mathcal{B}(\boldsymbol{u}_0, r)$. Intuitively, the radius r controls the size of the localized range, and thus mediates the trade-off between computational efficiency and partition coverage. A smaller radius limits the search to fewer partitions, reducing the number of points and hyper-planes considered. However, if the radius is too small, the localized candidate sets may contain too few points to construct a valid question. In contrast, a larger radius covers more partitions but increasingly resembles global processing, diminishing the benefits of localization.

We therefore control the size of $\mathcal{R}_l$ by introducing a concept called *selection ratio* ρ, which represents the expected fraction of partitions covered by the localized range. Assuming that partitions are approximately uniformly distributed over the $(d-1)$-dimensional utility range, ρ can be approximated by the ratio between the volume of the local $(d-1)$-dimensional ball and that of the utility range. This yields the following result.

Lemma 7. *Assume that all data points are uniformly distributed on the spherical surface. Given a target selection ratio ρ, if we set the radius of the localized hyper-sphere as*

$$r = \left(\rho \cdot \frac{\sqrt{d}}{(d-1)!} \cdot \frac{\Gamma\left(\frac{d-1}{2}+1\right)}{\pi^{(d-1)/2}}\right)^{\frac{1}{d-1}},$$

the expected number of partitions falling within the hyper-sphere is $\rho \cdot n_i$, where n_i denotes the size of $\mathcal{D}_i$.

Using ρ instead of a fixed radius provides a dimension-aware way to control the expected number of partitions. If the resulting radius is too small so that each localized candidate set contains only its anchor point and cannot yield a point pair for constructing a question, we can double r and repeat until at least one dataset produces a valid candidate hyper-plane.

For the second issue, we can solve an optimization problem to check if $\Theta_{\boldsymbol{q}}$ intersects a hyper-sphere $\mathcal{B}(\boldsymbol{u}_0, r)$:

$$\begin{aligned} \text{Minimize} \quad & |\boldsymbol{u} - \boldsymbol{u}_0|^2 \\ \text{Subject to} \quad & \boldsymbol{u} \in \Theta_{\boldsymbol{q}} \cap \mathcal{R}. \end{aligned}$$

This problem computes the smallest squared distance $|\boldsymbol{u}-\boldsymbol{u}_0|^2$ from the anchor $\boldsymbol{u}_0$ to a utility vector in $\Theta_{\boldsymbol{q}} \cap \mathcal{R}$. If $|\boldsymbol{u}-\boldsymbol{u}_0| \leq r$, then there exists a utility vector in $\Theta_{\boldsymbol{q}} \cap \mathcal{R}$ that lies inside the hyper-sphere, and hence we conclude $\Theta_{\boldsymbol{q}} \cap \mathcal{R}_l \neq \emptyset$.

Although this problem can be formulated as a Quadratic Programming (QP) problem, solving it is computationally expensive. To reduce the overhead, we instead employ an inexpensive vertex-based test as a sufficient condition. If at least one extreme point of $\Theta_{\boldsymbol{q}}$ lies inside $\mathcal{R}_l$, then $\Theta_{\boldsymbol{q}} \cap \mathcal{R}_l$ is non-empty, and $\boldsymbol{q}$ can be directly added to the localized candidate set $\mathcal{L}_i$. While this approach efficiently identifies intersecting partitions, it may miss some of them. Nevertheless, such omissions do not affect the correctness of the final result (see Theorem 5), as any missed partitions will be examined in subsequent Detect-and-Divide iterations if necessary.

Phase 2: Divide. With the localized sets $\mathcal{L}_i$ and their corresponding hyper-plane sets $\mathcal{H}_i$, we turn to the *divide* phase. The goal here is to interact with the user to refine the utility range $\mathcal{R}$. In each interactive round, we select a hyper-plane based on the pruning score. Based on the user's feedback, $\mathcal{R}$ will be refined to either $h^+_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$, making some partitions disjoint from $\mathcal{R}$. We prune these partitions and the corresponding points in $\mathcal{L}_i$ accordingly. This process continues until each $\mathcal{L}_i$ is reduced to exactly one point.

Then, we check whether the point left in $\mathcal{L}_i$ is the user's favorite point in $\mathcal{D}_i$. We introduce a concept *enclosed partition*, based on which we distinguish a class of points in each $\mathcal{L}_i$ that might be returned as the favorite in the current iteration.

Definition 4 (Enclosed Partitions)**.** *Given the localized set $\mathcal{L}_i$ at the beginning of the divide phase, a partition $\Theta_{\boldsymbol{p}}$ is said to be an enclosed partition if all its neighboring partitions $\Theta_{\boldsymbol{q}}$ have their corresponding points $\boldsymbol{q}$ included in $\mathcal{L}_i$.*

Here, enclosed partitions are identified only once, at the beginning of the divide phase, and remain designated as such throughout subsequent interaction rounds, even if some of their p-neighbors are later removed from $\mathcal{L}_i$. Intuitively, an enclosed partition $\Theta_{\boldsymbol{p}}$ is fully surrounded by partitions corresponding to points in $\mathcal{L}_i$, thereby providing $\boldsymbol{p}$ with a complete view of its local neighborhood. Since all the information needed to evaluate $\boldsymbol{p}$ against its local competitors is available within the localized candidate set, it is possible to conclude whether $\boldsymbol{p}$ can be returned as the user's local favorite point solely based on this local information, without examining any partitions outside. Formally, we have the following lemma.

Lemma 8. *If $\boldsymbol{p}^*$ is the only remaining point in $\mathcal{L}_i$ after several rounds of interactions, and its partition $\Theta_{\boldsymbol{p}^*}$ is an enclosed partition as identified at the beginning of the divide phase, then $\boldsymbol{p}^*$ is guaranteed to be the user's favorite point in $\mathcal{D}_i$.*

If Lemma 8 holds, we can return $\boldsymbol{p}^*$ directly as the user's favorite. Otherwise, some of its p-neighbors have not yet been discovered, indicating that potentially better points may exist outside the current localized candidate set. Then, a new *Detect-and-Divide* iteration is triggered with an updated anchor $\boldsymbol{u}_0$, guiding the localized search toward a more promising region.

The idea is to utilize the half-spaces induced by user feedback to provide directive information, since they provide hints about the location of the user's utility vector. Let $\mathbb{H}$ be the set of hyper-planes selected in the current iteration. We aggregate their unit normal vectors $\boldsymbol{n}$ to obtain a direction vector $\boldsymbol{v}$, i.e.,

$$\boldsymbol{v} = \frac{1}{|\mathbb{H}|} \sum_{h_{\boldsymbol{p},\boldsymbol{q}} \in \mathbb{H}} \boldsymbol{n}, \text{ where } \boldsymbol{n} = \begin{cases} \frac{\boldsymbol{p}-\boldsymbol{q}}{\|\boldsymbol{p}-\boldsymbol{q}\|}, & \text{if } \boldsymbol{p} \succ \boldsymbol{q}, \\ \frac{\boldsymbol{q}-\boldsymbol{p}}{\|\boldsymbol{q}-\boldsymbol{p}\|}, & \text{if } \boldsymbol{p} \prec \boldsymbol{q}. \end{cases}$$

After that, we compute the geometric center $\boldsymbol{u}^c$ of the only remaining partition $\Theta_{\boldsymbol{p}^*}$ and shoot a ray from $\boldsymbol{u}^c$ in the direction of $\boldsymbol{v}$ until it intersects the boundary of the global utility range $\mathcal{R}$. Denote the intersection by $\boldsymbol{u}^{\mathcal{R}}$. We take the midpoint between $\boldsymbol{u}^c$ and $\boldsymbol{u}^{\mathcal{R}}$ as the new anchor utility vector $\boldsymbol{u}_0$ for the next *Detect-and-Divide* iteration, i.e., $\boldsymbol{u}_0 = \frac{1}{2}(\boldsymbol{u}^{\mathcal{R}} + \boldsymbol{u}^c)$.

Example 7. *Continuing Example 6, suppose after several rounds of interaction, only one point $\boldsymbol{p}^*$ remains in $\mathcal{L}_i$. As shown in Figure 8, $\Theta_{\boldsymbol{p}^*}$ is not an enclosed partition since some of its neighboring partitions were not included in $\mathcal{L}_i$. We launch a new iteration with an updated anchor $\boldsymbol{u}_0$. After aggregating the normal vectors of the bounding hyper-planes (the*

Algorithm 2: Algorithm SHAREDQ

Input: A set of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$
Output: The user's favorite point $\boldsymbol{p}_i^*$ for each dataset $\mathcal{D}_i \in \mathrm{D}$
$\mathcal{R} \leftarrow \mathcal{U}$; $\boldsymbol{u}_0 \leftarrow$ a sampled utility vector in $\mathcal{R}$;
for $i = 1$ **to** m **do**
  Done$(\mathcal{D}_i) \leftarrow$ **false**;
while ***true*** **do**
  for $i = 1$ **to** m **do**
    if *Done*$(\mathcal{D}_i) =$ ***false*** **then**
      $\langle \mathcal{L}_i, \mathcal{H}_i \rangle \leftarrow$ **Detect**$(\mathcal{D}_i, \boldsymbol{u}_0)$;
  $\mathrm{L} = \{\mathcal{L}_1, \mathcal{L}_2, \ldots, \mathcal{L}_m\}$, $\mathrm{H} = \{\mathcal{H}_1, \mathcal{H}_2, \ldots, \mathcal{H}_m\}$;
  $\{\boldsymbol{p}_1^*, \ldots, \boldsymbol{p}_m^*\} \leftarrow$ **Divide**(L, H);
  for $i = 1$ **to** m **do**
    if $\boldsymbol{p}_i^*$ *is qualified to be returned by Lemma 8* **then**
      Done$(\mathcal{D}_i) \leftarrow$ **true**;
  if $\forall i \in \{1, \ldots, m\}$, *Done*$(\mathcal{D}_i) =$ ***true*** **then**
    return $\{\boldsymbol{p}_1^*, \boldsymbol{p}_2^*, \ldots, \boldsymbol{p}_m^*\}$;
  else
    Find a new utility vector $\boldsymbol{u}_0$;

Detect$(\mathcal{D}_i, \boldsymbol{u})$
$\mathcal{R}_l \leftarrow$ the localized utility range $\mathcal{R} \cap \mathcal{B}(\boldsymbol{u}, r)$;
Find the point $\boldsymbol{p}_i \in \mathcal{D}_i$ with the highest utility w.r.t. $\boldsymbol{u}$;
Initialize $\mathcal{L}_i \leftarrow \{\boldsymbol{p}_i\}$ and $\mathcal{H}_i \leftarrow \emptyset$;
while *there is an unvisited point in* $\mathcal{L}_i$ **do**
  $\boldsymbol{p} \leftarrow$ unvisited point in $\mathcal{L}_i$;
  Identify neighboring partitions of $\Theta_{\boldsymbol{p}}$ intersecting $\mathcal{R}_l$;
  Add the corresponding points into $\mathcal{L}_i$ if not included;
Construct $\mathcal{H}_i$ based on $\mathcal{L}_i$;
return $\langle \mathcal{L}_i, \mathcal{H}_i \rangle$;

Divide(L, H)
while $\exists i \in \{1, \ldots, m\}$ *s.t.* $|\mathcal{L}_i| > 1$ **do**
  Select a hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}} \in \mathrm{H}$ with the pruning score;
  Present the question based on $(\boldsymbol{p}, \boldsymbol{q})$ to the user;
  Update $\mathcal{R}$, $\mathcal{L}_i$ and $\mathcal{H}_i$ with the user's feedback;
return $\{\boldsymbol{p}_1^*, \ldots, \boldsymbol{p}_m^*\}$, *where* $\boldsymbol{p}_i^*$ *is the remaining point in* $\mathcal{L}_i$;

red lines) as $\boldsymbol{v}$, we shoot a ray (the red dashed line) from the center $\boldsymbol{u}^c$ of $\Theta_{\boldsymbol{p}^*}$ in the direction of $\boldsymbol{v}$ and hits the boundary of $\mathcal{R}$ at $\boldsymbol{u}^{\mathcal{R}}$. As shown in Figure 9, we take the midpoint between $\boldsymbol{u}^c$ and $\boldsymbol{u}^{\mathcal{R}}$ as the new anchor $\boldsymbol{u}_0$ and start the next iteration.

D. Summary

With these components in place, we present the pseudocode of algorithm SHAREDQ in Algorithm 2. It first initializes the global utility range $\mathcal{R}$ and the anchor utility vector $\boldsymbol{u}_0$ to the entire space $\mathcal{U}$ and a randomly sampled vector $\boldsymbol{u}_0 \in \mathcal{R}$, respectively (Line 3). After initializing the *Done* flag for each dataset to indicate that no dataset has been resolved (Lines 4–5), the algorithm enters an iterative loop that progressively identifies the user's true favorite point in each dataset through successive *Detect-and-Divide* iterations (Lines 6–18).

In each iteration, the algorithm first invokes the DETECT sub-procedure to explore the local neighborhood around $\boldsymbol{u}_0$ for every unresolved dataset $\mathcal{D}_i$. Specifically, we first identify a localized range $\mathcal{R}_l \subseteq \mathcal{R}$ (Line 20) and then compute the localized candidate set $\mathcal{L}_i$ and a candidate hyper-plane pool $\mathcal{H}_i$ (Lines 7–9) via recursive exploration (Lines 21–27), by only considering points whose partitions intersect $\mathcal{R}_l$. These local sets are aggregated into global collections L and H (Line 10).

Given the localized sets L and H, we invoke the DIVIDE sub-procedure (Line 11). It continually selects a hyper-plane from H based on the pruning score and interacts with the user (Lines 31–32). According to the user feedback, $\mathcal{R}$, $\mathcal{L}_i$ and $\mathcal{H}_i$ are updated accordingly. This process continues until there is only a single point p_i^* in each $\mathcal{L}_i$ (Lines 30). The algorithm then checks whether these points can be returned as the user's local favorite point in each dataset by Lemma 8 (Lines 12–14).

If all m datasets are flagged as resolved, the algorithm terminates and returns those points as the solution (Lines 15–16). Otherwise, a new anchor utility vector $\boldsymbol{u}_0$ is identified to launch the next *Detect-and-Divide* iteration. (Lines 17–18).

Analysis. The following two theorems prove the correctness and the upper bound on the number of questions, respectively.

Theorem 5. *Assume that the user's feedback are consistent with his/her utility vector $\boldsymbol{u}^*$.* SHAREDQ *terminates after finite interactions and returns the favorite point in every dataset.*

Theorem 6. *Assume that the points in each dataset are uniformly distributed in the space. If the detect phase is executed k times, algorithm* SHAREDQ *will identify the user's favorite point of each dataset within $O\left(k \log\left(\frac{r^{d-1}}{V_{min}}\right)\right)$ interactive rounds where V_{min} is the minimum volume of any partition.*

VII. EXPERIMENTS

We conducted experiments on a machine with Intel(R) Xeon(R) Platinum 8259CL CPUs at 2.50GHz and 64GB RAM. All programs were implemented in Python.

Datasets. We conducted experiments on both synthetic and real datasets. Synthetic datasets include *correlated*, *independent*, and *anti-correlated* distributions, generated using the standard skyline generator [4], [42], [43], [44]. For real datasets, we used *Google Play Store* [45] and *USA Real Estate* [46], containing $1,188,874$ and $1,254,996$ tuples, respectively. We partitioned Google Play Store by application category and USA Real Estate by the first digit of the five-digit ZIP code, yielding 10 distinct sub-datasets for each source. To exhibit heterogeneous attribute distributions, we considered different variants of real datasets with dimensionalities ranging from 2 to 4. All attributes are normalized into $(0, 1]$.

Baselines. We compared four baselines, UH-RANDOM [22], UH-SIMPLEX [22], RH [15], and HD-PI [15], following the adaptation in Section IV. Originally, UH-RANDOM and UH-SIMPLEX aim to find a point whose *regret ratio* [22] is below a predefined threshold ϵ. To guarantee that this point is the user's favorite point, we set $\epsilon = 0$. *RH* and HD-PI are traditionally designed to return the user's top-k favorite points; to align with our formulation, we restrict its output by setting $k = 1$.

Parameter Setting. We evaluated the performance of our algorithms by varying: (1) the selection ratio ρ in the DETECT algorithm (Section VI-C); (2) the parameters α and β, which measure the difference between two candidate hyper-planes (Section VI-B); (3) the number m of datasets; (4) the dimensionality d; (5) the size N of each dataset; and (6) the proportions of the three synthetic distributions (*correlated*: *anti-correlated*: *independent*). Unless stated explicitly, the default parameters for synthetic datasets are: $m = 10$, $N = 100,000$, and $d = 4$. The default distribution proportion is $3 : 3 : 4$.

Measurement. Since all algorithms are exact, i.e., they exactly identify the user's favorite point in each dataset, we evaluated each algorithm using the following two measurements: (1) *Execution time*, defined as the total computational time required for the entire interactive process; and (2) *Number of questions*, which represents the total number of interaction rounds needed to identify the user's favorite point in each dataset. To ensure reliability, we ran each algorithm 10 times with randomly generated utility vectors and reported the average.

A. Performance Study of Our Algorithm

We first evaluated the impact of various parameters on the performance of algorithm SHAREDQ, focusing primarily on the execution time and the number of questions asked.

Parameter ρ. We begin by studying the selection ratio ρ. As shown in Figure 10, although the numbers of questions fluctuate with different values of ρ, a larger value of ρ exhibits a longer execution time. Taking this into account, we adopted $\rho = 0.1$ as the default setting for the remaining experiments.

Parameter α/β. Next, we evaluated the impact of parameters α and β. Note that scaling both parameters by the same constant does not change how any candidate hyper-plane is evaluated; only their ratio matters. We therefore treat α/β as a single parameter in this experiment. As shown in Figure 11, SHAREDQ achieves the minimum number of questions while maintaining a highly competitive execution time when $\alpha/\beta = 1$. Thus, we set $\alpha = \beta = 1$ for all subsequent experiments.

Ablation Study. We conducted an ablation study to evaluate the effectiveness of the PCA-based hyper-plane reduction and

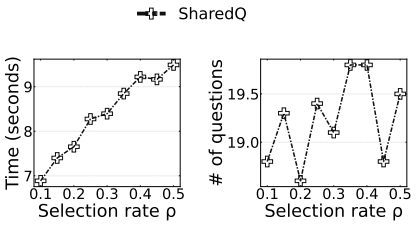


Figure 10: Parameter ρ

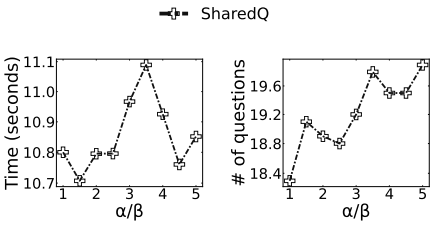


Figure 11: Parameter α/β

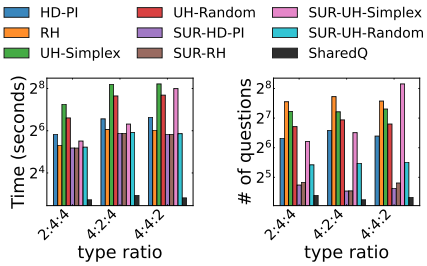


Figure 12: Different Distributions

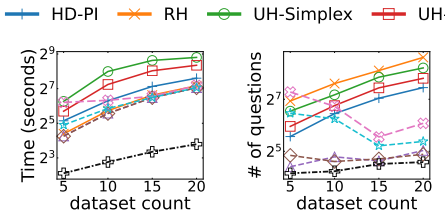


Figure 13: Different # of Datasets

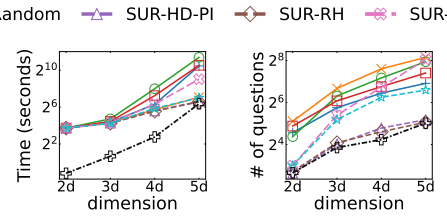


Figure 14: Vary d

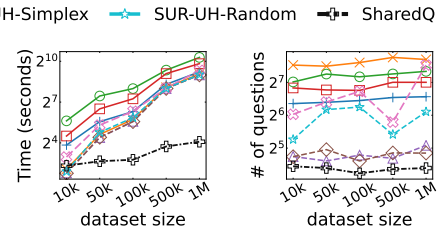
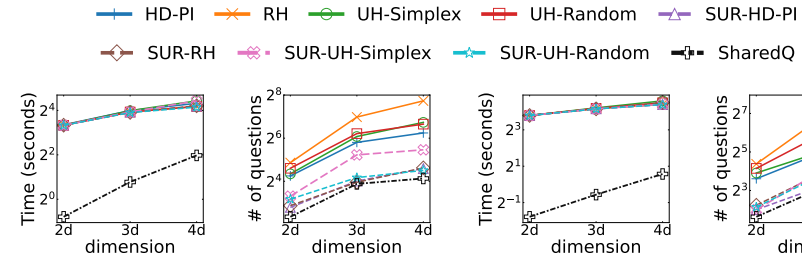
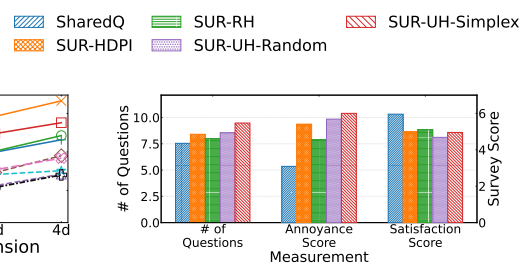


Figure 15: Different Dataset Sizes



(a) Google Play Store



(b) USA Real Estate

Figure 17: Real Datasets

Figure 18: User Study

Table IV: Ablation study

Configuration	# of questions	Time (seconds)
None	**19.5**	1113.372
PCA only	20.1	1008.386
Detect-and-Divide only	21.5	6.264
Full	20.0	**5.841**

Detect-and-Divide strategies. As shown in Table IV, both strategies improve computational efficiency. PCA-based hyperplane reduction alone reduces execution time by 9.4%, with only a slight increase in the number of questions. This justifies that the approximation reduces the time cost of question selection while keeping the interaction burden comparable. Better still, Detect-and-Divide further reduces execution time: together, the two strategies reduce execution time by 99.5% relative to the configuration with both disabled, while increasing the average number of questions by only 2.6%, highlighting the complementary benefits of the two strategies.

B. Performance on Synthetic Datasets

We proceed to compare our algorithms, SHAREDQ and the SUR variants (SUR-UH-RANDOM, SUR-UH-SIMPLEX, SUR-RH, SUR-HD-PI) with baselines. Unlike SHAREDQ, which requires no data preprocessing, both baselines and their SUR variants require the raw data to be preprocessed into skyline points. To ensure a fair comparison, we include this preprocessing overhead in their total execution times.

The distribution proportions of datasets. Figure 12 shows the performance of the algorithms under varying distribution proportions. One can observe that the efficiency of SUR variants is volatile: since they propagate refined utility ranges sequentially, the effectiveness depends heavily on whether early-explored datasets can offer high-quality constraints to prune more points before interaction. In contrast, algorithm SHAREDQ effectively mitigates these coupled limitations by globally optimizing shared hyper-planes across all datasets simultaneously. Thus, it demonstrates remarkable robustness and performance consistency across all distribution proportions, consistently outperforms all alternative approaches.

The number m of datasets. Figure 13 reports the result by varying the number m of datasets from 5 to 20, with equal proportions of each distribution. While a larger m naturally adds computational overhead, the number of question asked scales slowly. When m grows, the performance gap between the SUR-enhanced algorithms and sequential baselines widens significantly. SHAREDQ further extends this advantage, con-

sistently outperforming all baselines across the tested settings by reducing the number of questions by 50%–87% and cutting the total execution time to less than one-tenth of the baselines.

The dimensionality d. In Figure 14, we varied the dimensionality d from 2 to 5. As expected, an increase in dimensionality inevitably leads to both prolonged execution times and more questions for all evaluated methods. Nevertheless, our SHAREDQ algorithm consistently achieves significantly lower execution times, particularly when $d \leq 4$. In terms of the number of questions, SHAREDQ also establishes a clear superiority: compared with the best-performing SUR variant, SUR-HD-PI, it reduces the number of questions by 2–9, with an even larger advantage over the sequential baselines.

The size N of each dataset. Figure 15 studies the impact of dataset size N, varied from 10K to 1M. Compared to sequential baselines, the SUR-enhanced algorithms reduce the number of questions by nearly 65%, whereas SHAREDQ achieves an over 75% reduction. To improve efficiency, SHAREDQ limits its exploration to a localized subset of each dataset in a single round to avoid the costly computation of global partitions over all points. Besides, rather than expanding to a fixed number of partitions, it utilizes a localized hyper-sphere as the sub-range. These design choices contribute to SHAREDQ's superior execution time: it runs much faster than all competing methods when the dataset scales up, e.g., when $N \geq 50,000$.

C. Performance on Real Datasets

The performance evaluation on real datasets is shown in Figure 17. Consistent with previous experiments, with a larger dimensionality, all algorithms ask more questions and take longer times. Nevertheless, SHAREDQ runs substantially faster than other methods, and both SHAREDQ and the SUR-enhanced algorithms ask much fewer questions compared to the sequential baselines across the reported configurations, e.g., on Google Play Store (resp. USA Real Estate), SHAREDQ reduces the number of questions by 73.2% to 91.8% (resp. 71.8% to 93.1%) and total execution time by 78.2% to 94.3% (resp. 93.0% to 97.9%) compared to the sequential baselines.

D. User Study

To evaluate the practical usability of our framework and its resilience to real-world human factors (e.g., interaction fatigue and user errors), we conducted a user study on a used car dataset [47]. Similar to the real-world datasets, we partitioned this dataset by the state of sale, retaining 1,000 cars per sub-dataset. Each car is described by 4 attributes: price, year of purchase, odometer, and car condition. According to earlier results, the sequential baselines, UH-RANDOM [22], UH-SIMPLEX [22], RH [15], and HD-PI [15], ask excessive numbers of questions; this puts an unreasonable cognitive burden on participants. Thus we exclude them from this user study. We randomized the order of algorithms to avoid ordering bias.

We recruited 30 participants for the user study and report their average. Since asking a participant to examine thousands of tuples for a true favorite is impractical, reliable ground truths are difficult to obtain. Alternatively, each algorithm was evaluated based on three metrics following the setting in [7], [48], [49]: (1) The number of questions asked during the interaction. (2) Annoyance score: A subjective rating from 1 to 7 indicating the participant's level of fatigue or boredom during the interaction (1 denotes the least bored, and 7 denotes the most bored). (3) Satisfaction Score: A subjective rating from 1 to 7 reflecting the participant's satisfaction with the final returned tuples for each dataset (1 denotes the least satisfied, and 7 denotes the most satisfied). Intuitively, these three metrics measure the user effort, interaction fatigue, and satisfaction with the resulting choices, respectively.

Figure 18 shows the results. SHAREDQ requires the fewest user interactions, averaging only 7.5 questions per session, whereas the SUR-enhanced variants demand between 8.0 and 9.5 questions. Furthermore, SHAREDQ achieves the lowest Annoyance Score (3.1) and the highest Satisfaction Score (6.0), while the others incur annoyance scores exceeding 4.6 and satisfaction scores below 5.1 (with SUR-RH performing the best). These results demonstrate that SHAREDQ provides a better user experience by effectively mitigating interaction fatigue while maximizing user satisfaction.

E. Summary

The experiments demonstrate the superiority of our methods over best-known existing ones: (1) Efficiency and Effectiveness: Our algorithms substantially reduce both interaction overhead and execution time compared with existing approaches (e.g. on the $4D$ dataset, sequential baselines ask at least 87.3 questions, whereas our SUR-enhanced variants ask only 24.4. SHAREDQ widens this gap, requiring just 18.8 questions. (2) Scalability: Our algorithms exhibit robust scalability w.r.t. both dataset size and the number of datasets. (e.g. with 20 datasets, the baselines require at least 174.7 questions, whereas our SUR variants and SHAREDQ require only 29.5 and 24.0 questions, respectively). (3) Real-World Applicability: We deliver outstanding performance in practical deployment (e.g. on the $4D$ real-world *Google Play Store* dataset, SHAREDQ achieves an over 75% reduction in the number of questions compared to the traditional baselines).

VIII. CONCLUSION

In this paper, we study the problem of interactive query over multiple datasets, whose goal is to identify a user's favorite tuple in each dataset while minimizing the interaction overhead. Existing methods typically process datasets independently, leading to redundant interactions and discarding information learned from earlier datasets. To address this limitation, we introduce a shared utility range (SUR) mechanism, which transfers preference information across datasets, and propose SHAREDQ, an algorithm that selects informative questions jointly across datasets. SHAREDQ combines a pruning strategy with a Detect-and-Divide procedure to reduce both interaction and computational costs. Extensive experiments demonstrate the effectiveness and efficiency of our proposed algorithms.

For future work, we plan to extend our method to tolerate feedback errors and accommodate user preference drift.

IX. AI-Generated Content Acknowledgment

AI tools were used to assist with language polishing and grammar checking, as well as the implementation and review of experimental code. All AI-assisted content and code were carefully reviewed and verified by the authors, who take full responsibility for the correctness and originality of this work.

References

[1] S. Chester, A. Thomo, S. Venkatesh, and S. Whitesides, "Computing k-regret minimizing sets," *Proceedings of the VLDB Endowment*, vol. 7, no. 5, pp. 389–400, 2014.

[2] I. F. Ilyas, G. Beskales, and M. A. Soliman, "A survey of top-k query processing techniques in relational database systems," *ACM Computing Surveys (CSUR)*, vol. 40, no. 4, pp. 1–58, 2008.

[3] K. Zheng, H. Su, B. Zheng, S. Shang, J. Xu, J. Liu, and X. Zhou, "Interactive top-k spatial keyword queries," in *2015 IEEE 31st international conference on data engineering (ICDE)*. IEEE, 2015, pp. 423–434.

[4] S. Borzsony, D. Kossmann, and K. Stocker, "The skyline operator," in *2001 IEEE 17th international conference on data engineering (ICDE)*. IEEE, 2001, pp. 421–430.

[5] M. Xie, R. C.-W. Wong, J. Li, C. Long, and A. Lall, "Efficient k-regret query algorithm with restriction-free bound for any dimensionality," in *Proceedings of the 2018 International Conference on Management of Data*. Association for Computing Machinery, 2018, p. 959–974.

[6] J. Lee, G.-w. You, S.-w. Hwang, J. Selke, and W.-T. Balke, "Interactive skyline queries," *Information Sciences*, vol. 211, pp. 18–35, 2012.

[7] L. Qian, J. Gao, and H. Jagadish, "Learning user preferences by adaptive pairwise comparison," *Proceedings of the VLDB Endowment*, vol. 8, no. 11, pp. 1322–1333, 2015.

[8] D. Mindolin and J. Chomicki, "Discovering relative importance of skyline attributes," *Proceedings of the VLDB Endowment*, vol. 2, no. 1, pp. 610–621, 2009.

[9] B. Jiang, J. Pei, X. Lin, D. W. Cheung, and J. Han, "Mining preferences from superior and inferior examples," in *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2008, pp. 390–398.

[10] W.-T. Balke, U. Güntzer, and C. Lofi, "Eliciting matters–controlling skyline sizes by incremental integration of user preferences," in *International Conference on Database Systems for Advanced Applications*. Springer, 2007, pp. 551–562.

[11] J. Zheng and C. Chen, "Sorting-based interactive regret minimization," in *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data*. Springer, 2020, pp. 473–490.

[12] D. Nanongkai, A. D. Sarma, A. Lall, R. J. Lipton, and J. Xu, "Regret-minimizing representative databases," *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 1114–1124, 2010.

[13] G. Zhang, N. Tatti, and A. Gionis, "Finding favourite tuples on data streams with provably few comparisons," in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 3229–3238.

[14] W. Wang, V. J. Wei, M. Xie, D. Jiang, L. Fan, and H. Yang, "Interactive search with reinforcement learning," in *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 2025, pp. 1443–1455.

[15] W. Wang, R. C.-W. Wong, and M. Xie, "Interactive search for one of the top-k," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 1920–1932.

[16] W. Wang, R. C.-W. Wong, H. Jagadish, and M. Xie, "Reverse regret query," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 4100–4112.

[17] M. Zehlike, F. Bonchi, C. Castillo, S. Hajian, M. Megahed, and R. Baeza-Yates, "Fa* ir: A fair top-k ranking algorithm," in *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, 2017, pp. 1569–1578.

[18] B. Mirzasoleiman, A. Karbasi, R. Sarkar, and A. Krause, "Distributed submodular maximization: Identifying representative elements in massive data," *Advances in Neural Information Processing Systems*, vol. 26, 2013.

[19] K. Yao, J. Li, G. Li, and C. Luo, "Efficient group top-k spatial keyword query processing," in *Asia-Pacific Web Conference*. Springer, 2016, pp. 153–165.

[20] T. Deng and W. Fan, "On the complexity of query result diversification," *Proceedings of the VLDB Endowment*, vol. 6, no. 8, pp. 577–588, 2013.

[21] https://www2.crs.cuhk.edu.hk/sc/maics/scholarships-and-awards/.

[22] M. Xie, R. C.-W. Wong, and A. Lall, "Strongly truthful interactive regret minimization," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 281–298.

[23] I. Bartolini, P. Ciaccia, and M. Patella, "Domination in the probabilistic world: Computing skylines for arbitrary correlations and ranking semantics," *ACM Transactions on Database Systems (TODS)*, vol. 39, no. 2, pp. 1–45, 2014.

[24] G. Salton, "Automatic text processing: The transformation, analysis, and retrieval of," *Reading: Addison-Wesley*, vol. 169, 1989.

[25] P. Ciaccia, "Feedbackbypass: A new approach to interactive similarity query processing," 2001.

[26] D. Nanongkai, A. Lall, A. Das Sarma, and K. Makino, "Interactive regret minimization," in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, 2012, pp. 109–120.

[27] W. Wang and R. C.-W. Wong, "Interactive mining with ordered and unordered attributes," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2504–2516, 2022.

[28] W. Wang, R. C.-W. Wong, and M. Xie, "Interactive search with mixed attributes," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 2276–2288.

[29] K. G. Jamieson and R. Nowak, "Active ranking using pairwise comparisons," *Advances in Neural Information Processing Systems*, vol. 24, 2011.

[30] T.-Y. Liu *et al.*, "Learning to rank for information retrieval," *Foundations and Trends® in Information Retrieval*, vol. 3, no. 3, pp. 225–331, 2009.

[31] L. Maystre and M. Grossglauser, "Just sort it! a simple and effective approach to active preference learning," in *International Conference on Machine Learning*. PMLR, 2017, pp. 2344–2353.

[32] B. Eriksson, "Learning to top-k search using pairwise comparisons," in *Artificial Intelligence and Statistics*. PMLR, 2013, pp. 265–273.

[33] E. Dushkin and T. Milo, "Top-k sorting under partial order information," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 1007–1019.

[34] Y. Jia, H. Wang, S. Guo, and H. Wang, "Pairrank: Online pairwise learning to rank by divide-and-conquer," in *Proceedings of the web conference 2021*, 2021, pp. 146–157.

[35] J. Liao, A. Lall, M. Ogihara, and R. Wong, "High-dimensional regret minimization," *arXiv preprint arXiv:2512.24078*, 2025.

[36] B. Tang, K. Mouratidis, and M. L. Yiu, "Determining the impact regions of competing options in preference space," in *Proceedings of the 2017 ACM SIGMOD International Conference on Management of Data*, 2017, pp. 805–820.

[37] Y.-C. Chang, L. Bergman, V. Castelli, C.-S. Li, M.-L. Lo, and J. R. Smith, "The onion technique: Indexing for linear optimization queries," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of data*, 2000, pp. 391–402.

[38] Anonymous, "Interactive query over multiple datasets," Tech. Rep., 2026. [Online]. Available: https://github.com/Welt000/Interactive-Query-over-Multiple-Datasets

[39] M. De Berg, O. Cheong, M. Van Kreveld, and M. Overmars, *Computational geometry: Algorithms and applications*. Springer Berlin Heidelberg, 2008.

[40] I. T. Jolliffe and J. Cadima, "Principal component analysis: a review and recent developments," *Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences*, vol. 374, no. 2065, p. 20150202, 2016.

[41] J. Shlens, "A tutorial on principal component analysis," *arXiv preprint arXiv:1404.1100*, 2014.

[42] G. Koutrika, E. Pitoura, and K. Stefanidis, "Preference-based query personalization," in *Advanced Query Processing: Volume 1: Issues and Trends*. Springer, 2013, pp. 57–81.

[43] D. Papadias, Y. Tao, G. Fu, and B. Seeger, "Progressive skyline computation in database systems," *ACM Transactions on Database Systems (TODS)*, vol. 30, no. 1, pp. 41–82, 2005.

[44] Y. Gao, Q. Liu, B. Zheng, L. Mou, G. Chen, and Q. Li, "On processing reverse k-skyband and ranked reverse skyline queries," *Information Sciences*, vol. 293, pp. 11–34, 2015.

[45] "Google play store apps," https://www.kaggle.com/datasets/gauthamp10/google-playstore-apps, 2019.

[46] "Usa real estate dataset," https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset, 2023.

[47] "Used cars dataset," https://www.kaggle.com/datasets/austinreese/craigslist-carstrucks-data, 2021.
[48] Y. Tao, J. Zhang, D. Papadias, and N. Mamoulis, "An efficient cost model for optimization of nearest neighbor search in low and medium dimensional spaces," *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 10, pp. 1169–1184, 2004.
[49] M. Xie, T. Chen, and R. C.-W. Wong, "Findyourfavorite: An interactive system for finding the user's favorite tuple in the database," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 2017–2020.
[50] R. T. Rockafellar, *Convex analysis*. Princeton university press, 1997, vol. 28.

X. Table

The frequently used notations are shown in Table V.

XI. Proof

Proof of Theorem 1. For any individual dataset $\mathcal{D}_i$, any algorithm driven by pairwise comparison can be modeled as a binary tree. Each leaf corresponds to a point, and each internal node corresponds to a question asked to a user. Since $\mathcal{D}_i$ contains n_i points, the decision tree must have at least n_i leaves, yielding a height of $\Omega(\log_2 n_i)$ to find the desired point. To complete the query task over all m datasets, the algorithm is lower-bounded by the largest dataset. Even assuming perfect parallelism across datasets, the total interaction rounds cannot be fewer than the maximum height among all individual trees, which is $\Omega(\max_{i\in[1,m]} \log_2 n_i) = \Omega(\log_2 N)$. □

Proof of Theorem 2. It has been proven that identifying a user's favorite point within a single dataset of size n in $O(n)$ interactive rounds in worst case and $O(\deg_{\max} \cdot \sqrt[d]{n})$ on average for algorithm UH-Simplex [22]. Since the baseline method processes each of the m datasets independently and sequentially without any information transfer, the total number of questions is bounded by the sum of their individual maximum requirements. Therefore, the overall upper bound is derived as:

$$\begin{cases} O\left(\sum_{i=1}^{m} n_i\right) & \text{worst case} \\ O\left(\deg_{\max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_i}\right) & \text{on average} \end{cases}$$

□

Proof of Lemma 2. Each user feedback compares two points and, by Lemma 1, induces a half-space that contains the user's utility vector $\boldsymbol{u}^*$. Since $\mathcal{R}$ is obtained by intersecting such half-spaces, we have $\boldsymbol{u}^* \in \mathcal{R}$. The utility vector $\boldsymbol{u}^*$ is shared across all datasets in the IMD problem. Therefore, every half-space collected from a dataset is also valid for other datasets, and $\mathcal{R}$ can be safely reused. □

Proof of Lemma 3. According to the definition of $h_{\boldsymbol{q},\boldsymbol{p}}$, we have $f_{\boldsymbol{u}}(\boldsymbol{q}) > f_{\boldsymbol{u}}(\boldsymbol{p})$ for any $\boldsymbol{u} \in h^+_{\boldsymbol{q},\boldsymbol{p}}$. Given that $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$, any utility vector $\boldsymbol{u} \in \mathcal{R}$ must inherently reside within $h^+_{\boldsymbol{q},\boldsymbol{p}}$, which translates to $f_{\boldsymbol{u}}(\boldsymbol{q}) > f_{\boldsymbol{u}}(\boldsymbol{p})$. Consequently, the utility of $\boldsymbol{p}$ can never surpass that of $\boldsymbol{q}$ for any utility vector within $\mathcal{R}$. □

Proof of Lemma 4. Since $\mathcal{R}$ is formed by the intersection of several half-spaces, it is *convex* [39]. Let $\mathcal{E} = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_k\}$ denote the finite set of all extreme points of $\mathcal{R}$, where $\boldsymbol{e}_i \in h^+_{\boldsymbol{q},\boldsymbol{p}}$ holds for all $\boldsymbol{e}_i \in \mathcal{E}$. Since $\mathcal{R}$ is convex, according to the Krein-Milman theorem [50], any utility vector $\boldsymbol{u} \in \mathcal{R}$ can be represented as a convex combination of its extreme points $\mathcal{E}$. That is, there exist non-negative coefficients $\lambda_1, \lambda_2, \ldots, \lambda_k$ such that:

$$\boldsymbol{u} = \sum_{j=1}^{k} \lambda_j \boldsymbol{e}_j, \quad \text{where } \sum_{j=1}^{k} \lambda_j = 1 \text{ and } \lambda_j \geq 0. \tag{1}$$

Consequently, for any $\boldsymbol{u} \in \mathcal{R}$, the formula can be expanded as:

$$\boldsymbol{u} \cdot (\boldsymbol{q} - \boldsymbol{p}) = \left(\sum_{j=1}^{k} \lambda_j \boldsymbol{e}_j\right) \cdot (\boldsymbol{q} - \boldsymbol{p}) = \sum_{j=1}^{k} \lambda_j \left[\boldsymbol{e}_j \cdot (\boldsymbol{q} - \boldsymbol{p})\right].$$

Given that $\boldsymbol{e}_j \in h^+_{\boldsymbol{q},\boldsymbol{p}}$ (i.e., $\boldsymbol{e}_j \cdot (\boldsymbol{q} - \boldsymbol{p}) > 0$) and $\lambda_j \geq 0$ for all j, with at least one $\lambda_j > 0$, it follows that:

$$\boldsymbol{u} \cdot (\boldsymbol{q} - \boldsymbol{p}) > 0,$$

which implies $\boldsymbol{u} \in h^+_{\boldsymbol{q},\boldsymbol{p}}$. Since this relation holds universally for all $\boldsymbol{u} \in \mathcal{R}$, we successfully establish the conclusion $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$. □

Proof of Theorem 4. As established in Section VI-B, the m datasets collectively generate $O(\sum_{i=1}^{m} n_i^2)$ candidate hyper-planes and $O(\sum_{i=1}^{m} n_i)$ partitions. In the conventional approach, one must exhaustively evaluate the positional relationship between every hyper-plane and every partition. According to [27], verifying this relationship requires checking all v extreme points of a partition/Consequently, the overall complexity of the traditional approach is:

$$O\left(\left(\sum_{i=1}^{m} n_i^2\right) \cdot \left(\sum_{i=1}^{m} n_i\right) \cdot v\right)$$

.

In our PCA-based method, computing the middle point of each partition requires $O(\sum_{i=1}^{m} n_i \cdot v)$ operations. Centering these points and calculating their mean is achieved in $O(\sum_{i=1}^{m} n_i \cdot d)$, while computing the covariance matrix introduces a complexity of $O(\sum_{i=1}^{m} n_i \cdot d^2)$. Next, the eigenvalue decomposition adds a bound of $O(d^3)$ per dataset, yielding an overall cost of $O(m \cdot d^3)$ for this step. Ultimately, evaluating the positional relationship of the final m optimal hyper-planes against all $O(\sum_{i=1}^{m} n_i)$ partitions necessitates $O(\sum_{i=1}^{m} n_i \cdot m \cdot v)$ operations. Summing these individual bounds, the overall time complexity of our PCA method reduces to:

$$O\left(\sum_{i=1}^{m} n_i(d^2 + m \cdot v) + m \cdot d^3\right).$$

□

Proof of Theorem 3. Prior work establishes that the UH-Simplex algorithm guarantees an interactive upper bound of $O(n)$ in the worst case, and $O(\deg_{\max} \cdot \sqrt[d]{n})$ on average, for a single dataset of size n [22]. Under the assumption that all data points are uniformly distributed on the spherical

Table V: Frequently Used Notations

Notation	Definition
D and $\mathcal{D}_i$	All datasets and the i-th dataset.
m and n_i	Number of datasets and points in $\mathcal{D}_i$.
$\boldsymbol{p}$ and d	A point and dimensionality.
$\boldsymbol{u}$ and $\mathcal{R}$	Utility vector and global utility range.
$\Theta_{\boldsymbol{p}}$	Partition of point $\boldsymbol{p}$.
$h_{\boldsymbol{p},\boldsymbol{q}}$	Hyper-plane of $\boldsymbol{p}$ and $\boldsymbol{q}$.
$h^+_{\boldsymbol{p},\boldsymbol{q}}/h^-_{\boldsymbol{p},\boldsymbol{q}}$	Positive/negative half-space of $h_{\boldsymbol{p},\boldsymbol{q}}$.

surface, when a refined utility range $\mathcal{R}_{i-1}$ is propagated to $\mathcal{D}_i$ as a global constraint, the expected number of surviving candidate points in $\mathcal{D}_i$ can be estimated by the volume ratio as $n_{\mathcal{R}_{i-1}} = \lceil n_i \cdot \frac{V(\mathcal{R}_{i-1})}{V(\mathcal{U})} \rceil$. Since the initial utility range $\mathcal{R}_0$ is defined as the entire space $\mathcal{U}$, we naturally have $n_{\mathcal{R}_0} = n_1$. By sequentially aggregating the interactive overhead across all datasets, we derive the overall complexity of the SUR-UH-SIMPLEX algorithm as:

$$\begin{cases} O\left(\sum_{i=1}^{m} n_{\mathcal{R}_{i-1}}\right) & \text{worst case} \\ O\left(\deg_{\max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_{\mathcal{R}_{i-1}}}\right) & \text{average case} \end{cases}$$

□

Proof of Lemma 5. For any utility vector $\boldsymbol{u} \in \Theta_{\boldsymbol{p}} \cap \mathcal{U}$, suppose that there exists a point $\boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\}$ whose utility is strictly higher that that of $\boldsymbol{p}$, i.e., $\boldsymbol{u}\cdot\boldsymbol{q} > \boldsymbol{u}\cdot\boldsymbol{p}$. By the definition of the partition, the premise $\boldsymbol{u} \in \Theta_{\boldsymbol{p}}$ implies that $\boldsymbol{u} \in h^+_{\boldsymbol{p},\boldsymbol{q}}$, which dictates $\boldsymbol{u}\cdot\boldsymbol{p} > \boldsymbol{u}\cdot\boldsymbol{q}$. This contradicts our assumption. Therefore, we can conclude that $\boldsymbol{p}$ yields the highest utility in $\mathcal{D}_i$ w.r.t. any $\boldsymbol{u} \in \Theta_{\boldsymbol{p}} \cap \mathcal{R}$. □

Proof of Lemma 6. By definition, for any $\boldsymbol{u} \in h^+_{\boldsymbol{p},\boldsymbol{q}}$, we have $f_{\boldsymbol{u}}(\boldsymbol{p}) > f_{\boldsymbol{u}}(\boldsymbol{q})$. Let $\boldsymbol{u}'$ be an arbitrary point lying on the boundary of $\Theta_{\boldsymbol{q}}$, we have $f_{\boldsymbol{u}'}(\boldsymbol{p}) = f_{\boldsymbol{u}'}(\boldsymbol{q})$. This implies that $\boldsymbol{u}'$ is also lies in the boundary of $\Theta_{\boldsymbol{q}}$, rendering $\Theta_{\boldsymbol{q}}$ non-empty. □

Proof of Lemma 7. Given that the utility vectors satisfy $\sum_{i=1}^{d} u_i = 1$ and $u_i \geq 0$, the global utility space is essentially a $(d-1)$-dimensional standard simplex embedded in $\mathbb{R}^d$. The $(d-1)$-dimensional volume of this utility space is known as $V_U = \frac{\sqrt{d}}{(d-1)!}$. Accordingly, the local neighborhood $\mathcal{B}(\boldsymbol{u}, r)$ restricted to this simplex is a $(d-1)$-dimensional hyper-ball, whose volume is given by $V_{\mathcal{B}}(r) = \frac{\pi^{(d-1)/2}}{\Gamma(\frac{d-1}{2}+1)} r^{d-1}$.

Under the assumption that all data points are uniformly distributed on the spherical surface, the volume of all partitions are the same. Thus, the expected selection ratio ρ is equivalent to the ratio of the volume of the local subset to the total volume of the utility space, i.e., $\rho = \frac{V_{\mathcal{B}}(r)}{V_U}$. By expanding this equation, we have:

$$\rho = \frac{\frac{\pi^{(d-1)/2}}{\Gamma(\frac{d-1}{2}+1)} r^{d-1}}{\frac{\sqrt{d}}{(d-1)!}}$$

Solving for r yields the desired approximation:

$$r = \left(\rho \cdot \frac{\sqrt{d}}{(d-1)!} \cdot \frac{\Gamma\left(\frac{d-1}{2}+1\right)}{\pi^{(d-1)/2}} \right)^{\frac{1}{d-1}}.$$

In this way, the expected number of partitions falling within the hyper-sphere $n_{\mathcal{B}}$ satisfies $\rho = \frac{n_{\mathcal{B}}}{n_i}$, i.e. $n_{\mathcal{B}} = \rho \cdot n_i$. □

Proof of Lemma 8. Suppose that $\boldsymbol{p}_i$ is not the user's favorite point. Since $\boldsymbol{p}_i$ remains a feasible candidate, while the user's true utility vector $\boldsymbol{u}$ lies outside $\Theta_{\boldsymbol{p}_i}$, $\mathcal{R}_l$ must intersect a neighbor of $\Theta_{\boldsymbol{p}_i}$. By the definition of the enclosed partition, the corresponding p-neighbor is included in $\mathcal{L}_i$ and cannot be safely pruned. This contradicts the assumption that $\boldsymbol{p}_i$ is the only remaining point in $\mathcal{L}_i$. Therefore, $\boldsymbol{p}_i$ is the user's favorite point in $\mathcal{D}_i$. □

Proof of Theorem 5. Each user feedback induces a half-space constraint consistent with $\boldsymbol{u}^*$. Therefore, $\boldsymbol{u}^*$ always remains in the global utility range $\mathcal{R}$. Moreover, a candidate point is pruned only if it cannot be optimal under any utility vector in the current global utility range $\mathcal{R}$. Thus, the user's true favorite point is never incorrectly pruned.

To prove termination, each informative comparison eliminates at least one candidate point. Since all datasets are finite and eliminated points cannot become feasible again, the number of informative comparisons is finite. In the worst case, SharedQ falls back to pairwise comparisons, requiring at most $n_i - 1$ additional rounds to identify the favorite point in each unresolved dataset $\mathcal{D}_i$. Therefore, SharedQ terminates after finitely many interactions.

For each dataset $\mathcal{D}_i$, SharedQ marks it as resolved only when its remaining point $\boldsymbol{p}_i$ corresponds to an enclosed partition. By Lemma 8, $\boldsymbol{p}_i$ must be the user's favorite point in $\mathcal{D}_i$. Since SharedQ terminates only when all datasets are resolved, every returned point is the user's favorite in its corresponding dataset. □

Proof of Theorem 6. During the DETECT phase, the algorithm identifies all candidate partitions enclosed within the localized hyper-sphere $\mathcal{B}(\boldsymbol{u}_0, r)$. Let $V_d(r) \propto r^{d-1}$ denote the volume of a d-dimensional hyper-sphere. Given the minimum partition volume V_{min}, the maximum number of candidate partitions within $\mathcal{B}(\boldsymbol{u}_0, r)$ is bounded by $O\left(\frac{r^{d-1}}{V_{min}}\right)$. Subsequently, in the DIVIDE phase, the strategically selected hyper-plane efficiently eliminates approximately half of the candidate partitions in each interactive round. Consequently, the number of questions required to isolate a single point in one stage is bounded by $O\left(\log\left(\frac{r^{d-1}}{V_{min}}\right)\right)$. Accumulating this overhead across k stages, the total number of interactive rounds is given by:

$$R = O\left(k \log\left(\frac{r^{d-1}}{V_{min}}\right)\right).$$

□